

Autoscaler gerarchico per la gestione di congestioni causate da fluttuazioni nel traffico in arrivo e nelle richieste

DI GENTILI EMANUELE

24/06/2026

Indice

1	Introduzione	4
1.1	Contesto	4
1.2	Caso di studio	4
2	Obiettivi	5
3	Il modello concettuale	6
3.1	Descrizione dei componenti del sistema	6
3.1.1	Il load controller	6
3.1.2	I server	6
3.2	Lo stato del sistema	6
3.3	Gli eventi nel sistema	6
4	Modello delle specifiche	7
4.1	I parametri del sistema	7
4.2	La modellazione del load controller	8
4.3	Fluttuazioni long-term	8
4.4	La variabile aleatoria iperesponenziale	8
4.5	Gli intervalli di confidenza	9
5	Modello computazionale	10
5.1	La gestione degli eventi	10
5.1.1	Evento di arrivo	10
5.1.2	Evento di completamento	11
5.2	Lo stato del sistema: la classe SystemState	12
5.2.1	Gestione della coda degli eventi: la classe EventCalendar	13
5.2.2	Comportamento dell'infrastruttura del sistema: le classi Infrastructure	13
5.2.3	Astrazione dei server: le classi Server	13
5.2.4	Astrazione dei job: le classi JobList e Job	13
5.3	Avanzamento dei job	14
5.4	Le variabili aleatorie	16
5.4.1	Gestione dei seed	16
5.4.2	La variabile aleatoria iperesponenziale	16
5.5	La configurazione del sistema e gli esperimenti	17
5.6	Raccolta di dati e logging	17
5.7	Calcolo degli intervalli di confidenza	17
6	Verifica	18
6.1	Domanda 1	18
6.2	Domanda 2	18
6.3	Domanda 3	18
6.4	Domanda 4	19
7	Validazione	20
7.1	Domanda 1	20
7.2	Domanda 2	21
7.3	Domanda 3	21
8	Studio del transitorio	23
8.1	Studio 1	23
8.2	Studio 2	24
8.3	Studio 3	24

9	Esperimenti	26
9.1	Esperimento 1	26
9.2	Esperimento 2	30
9.2.1	Analisi del sistema a basso carico	31
9.2.2	Analisi del sistema con carico medio	31
9.2.3	Analisi del sistema con carico alto	31
10	Introduzione al modello migliorativo	33
11	Modello concettuale migliorativo	34
11.1	Componenti	34
11.1.1	I Web Server	34
11.1.2	Il load controller	34
11.2	Gli eventi del sistema	34
12	Modello delle specifiche migliorativo	36
12.1	Inizializzazione del sistema	36
12.2	Modellazione dell'autoscaling orizzontale	36
12.2.1	Indicatori di scaling	36
12.2.2	Le decisioni di scaling	36
12.2.3	Tempi di accensione	37
13	Modello computazionale migliorativo	38
13.1	La gestione degli eventi	38
13.1.1	Evento di arrivo	38
13.1.2	Evento di completamento	38
13.1.3	La funzione che pianifica lo scaling	38
13.1.4	Evento richiesta di scaling out	38
13.1.5	Evento di scaling out	40
13.1.6	Evento di scaling in	40
13.2	La variabile aleatoria Normale	41
14	Validazione del modello migliorativo	42
14.1	Studio 1	42
14.2	Studio 2	43
15	Studio del transitorio	44
15.1	Studio 1	44
15.2	Studio 2	45
16	Esperimenti sul modello migliorativo	46
16.1	Studio 1	47
16.2	Studio 2	49
16.3	Studio 3	50
17	Conclusioni	52
17.1	La scelta della soglia $SI^{\max}$ nel modello base	52
17.2	L'impatto del tasso di arrivo sullo spike server	52
17.3	Configurazioni degli autoscaler nel modello migliorativo	52
17.4	Comportamento delle soglie durante il periodo di fluttuazione	52
18	Appendice	53
18.1	Validazione manuale dello stato del sistema	53
18.2	I test delle variabili aleatorie	53

1 Introduzione

1.1 Contesto

In questo caso di studio viene descritto un modello che simula un autoscaler gerarchico in grado di gestire le congestioni create dalle fluttuazioni del carico in arrivo e dalla domanda computazionale. Tali fluttuazioni interessano la maggior parte dei data center dei provider di servizi internet. A seconda del servizio fornito le fluttuazioni possono avere differenti intensità e scale. Questo comporta uno scenario molto variabile che rende l'identificazione del minimo numero di risorse, per raggiungere un obiettivo in termini di performance, un problema molto complesso. Infatti, sovradimensionando il sistema, si riescono a raggiungere gli obiettivi preposti, ma alcune risorse risultano poco utilizzate. Al contrario, sottodimensionando il sistema, è possibile utilizzare meno risorse, ma le performance peggiorano.

Per risolvere questi problemi, sono sempre più utilizzate delle tecniche di autoscaling orizzontale per allocare e deallocare dinamicamente i server. Tuttavia, l'accensione di un server richiede del tempo, rendendo questa soluzione non sostenibile nel caso di fluttuazioni a breve termine. A tal proposito, vengono utilizzate delle risorse che sono presenti sin dall'inizio, ma vengono considerate solamente in determinate situazioni di congestione. Il risultato finale sarà un **autoscaler gerarchico** che gestisce sia le fluttuazioni a breve che lungo termine.

1.2 Caso di studio

Il sistema è costituito da una risorsa primaria, il Web Server (WS). Il WS viene utilizzato per gestire le richieste fino alla sua saturazione. Nel momento in cui il WS si satura entra in gioco lo Spike Server (SS) che si occupa di servire le richieste al posto del WS finché la congestione non termina. Questo comportamento è garantito da un meccanismo di routing dinamico che sulla base dello stato del WS decide a chi inviare le richieste.

2 Obiettivi

Gli obiettivi di questo studio sono i seguenti:

1. Individuare il valore di $SI^{\max}$ che porta ad un tempo di risposta del sistema il più vicino possibile a 8 s.
2. Studiare l'impatto di diversi valori di $SI^{\max}$ sul tempo di risposta in funzione del tasso di arrivo del sistema.
3. Valutare diverse configurazioni per l'autoscaler basato sul tempo di risposta.
4. Valutare diverse diverse configurazioni per l'autoscaler basato sul numero di job nel sistema.
5. Confrontare i due meccanismi di autoscaling con il caso senza autoscaler orizzontale, durante un periodo di 900 s.

La configurazione dell'autoscaler è data da:

- Il valore di $SI^{\max}$.
- Le soglie per prendere decisioni di scaling orizzontale.

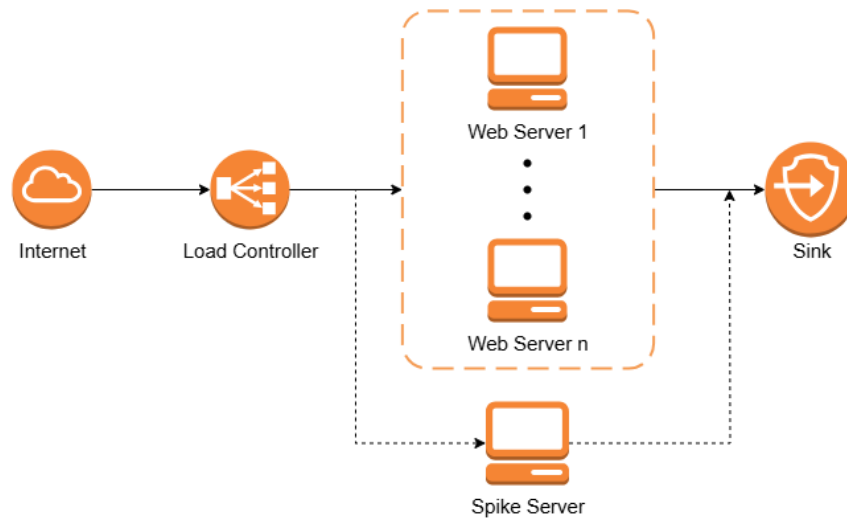


Figura 1. Il modello concettuale del sistema

3 Il modello concettuale

Una rappresentazione del modello concettuale è riportata in figura 1.

3.1 Descrizione dei componenti del sistema

3.1.1 Il load controller

Il load controller incapsula la logica di routing dinamico. Ovvero, decide se inviare le richieste al web server o allo spike server, in funzione dello stato del web server. In particolare, invia le richieste allo spike server solamente se il web server è congestionato. Quest'ultimo si dice congestionato se il numero di richieste pendenti supera una soglia scelta. Nel momento in cui questa condizione si verifica, le richieste verranno inoltrate tutte allo spike server. Quando il web server ha smaltito un numero di richieste sufficienti per tornare al di sotto della soglia, verrà nuovamente considerato dal meccanismo di routing dinamico.

3.1.2 I server

I server, sia spike che web, utilizzano uno scheduling di tipo processor sharing per servire le richieste assegnate dal load controller.

3.2 Lo stato del sistema

Le variabili di stato sono:

- Gli insiemi $\mathcal{J}_k(t)$ di richieste che il server k sta servendo al tempo t . Questi insiemi sono formati da coppie del tipo $(j, r_j(t))$ dove:
 - j è un identificatore per il job
 - $r_j(t)$: tempo di servizio rimanente del job j al tempo t .
- La capacità C_k del server k viene influenzata dalla politica di scheduling di tipo processor sharing. Ovvero, la capacità verrà suddivisa tra tutti i job che il server ha in carico al tempo t . Quindi la capacità del server k al tempo t è data da:

$$v_k(t) = \frac{C_k}{|\mathcal{J}_k(t)|}$$

Tra l'istante t e $t + \Delta t$ il tempo di servizio rimanente di ciascun job viene decrementato secondo la seguente regola:

$$r_j(t + \Delta t) = r_j(t) - v_k(t) \Delta t$$

Nell'appendice è presente l'esempio 4 che mostra l'evoluzione dello stato in un caso semplice.

3.3 Gli eventi nel sistema

Nel sistema possono verificarsi due tipi di evento:

- **Arrivo:** un job raggiunge il load controller e verrà assegnato ad un server.
- **Completamento:** il job servito lascerà il sistema.

4 Modello delle specifiche

4.1 I parametri del sistema

Nella tabella 1 sono riportati i parametri del sistema.

Osservazione 1. L'utilizzo della distribuzione iperesponenziale permette di:

- Avere una coerenza con lo studio del libro.
- Stimolare il sistema con delle fluttuazioni short-term per via della sua caratteristica di tipo heavy-tail.
- Sottoporre il sistema ad una maggiore variabilità di carico e vedere come si comporta durante intervalli di congestione.

Osservazione 2. Il tempo di servizio è il rapporto tra una variabile di tipo iperesponenziale ed una costante. Quindi anche il tempo di servizio seguirà una distribuzione di tipo iperesponenziale i cui parametri possono essere calcolati in questo modo:

$$E[S] = \frac{E[Z]}{C}$$

Dove Z indica la domanda dei job e C la capacità di un server. Invece il coefficiente di variazione è dato da:

$$C_S^2 = \frac{\text{Var}[S]}{E^2[S]}$$

Dove, posto $k = \frac{1}{C}$

- $\text{Var}[S] = \text{Var}[k Z] = k^2 \text{Var}[Z]$
- $E[S] = E[k Z] = k E[Z]$

Quindi:

$$\begin{aligned} C_S^2 &= \frac{k^2 \text{Var}[Z]}{k^2 E^2[Z]} \\ &= \frac{\text{Var}[Z]}{E^2[Z]} \\ &= C_Z^2 \end{aligned}$$

Osservazione 3. Gli arrivi al sistema seguono una distribuzione iperesponenziale. Tuttavia, la presenza del routing dinamico non permette di modellare ciascun server come una $H2/H2/1/PS$. Dunque, ciascun server sarà modellato come una $G/H2/1/PS$.

Parametro	Distribuzione	Parametri della distribuzione
Tempo di inter-arrivo	Iperesponenziale	Media 0.15 s, Coefficiente di variazione 4
Domande dei job	Iperesponenziale	Media 0.16 op., Coefficiente di variazione 4
Tempo di servizio	Iperesponenziale	Media 0.16 s, Coefficiente di variazione 4
Capacità	Deterministica	$1 \frac{\text{op.}}{s}$

Tabella 1. I parametri del sistema

4.2 La modellazione del load controller

Gli arrivi al sistema sono gestiti secondo una politica di tipo least-used, ovvero, quando arriva un job esso viene assegnato al server che ha il minor numero di richieste pendenti. Nel modello base essendoci un solo web server, il load controller manderà ad esso le richieste, fino a saturarlo. Il numero di job che coesistono contemporaneamente all'interno di un singolo web server viene indicato con SI (Spike Indicator). Questo valore viene utilizzato per decidere quando cominciare ad utilizzare lo spike server. In particolare, viene definita una soglia $SI^{\max}$, il cui superamento indica un momento di congestione del web server. Quindi, in generale, in un sistema in cui ci sono m web server, quando si verifica la condizione:

$$SI > m SI^{\max}$$

le richieste vengono inoltrate verso lo spike server finché la precedente condizione è vera.

4.3 Fluttuazioni long-term

Per simulare le fluttuazioni long-term è stato definito uno scenario in cui periodicamente, ogni 500s, si susseguono:

- Un periodo di 400s, durante il quale il tasso degli arrivi ha una media $\lambda = \frac{1}{0.15}$.
- Un periodo di 100s, durante il quale il tasso degli arrivi ha una media $\lambda_{\text{fast}} = \frac{4}{0.15}$.

4.4 La variabile aleatoria iperesponenziale

L'implementazione della variabile aleatoria iperesponenziale è funzione del suo coefficiente di variazione C^2 e della sua media $\frac{1}{\mu_X}$. Per costruire una variabile aleatoria iperesponenziale è necessario utilizzare:

- Due variabili aleatorie esponenziali con medie μ_1 e μ_2 .
- Una variabile aleatoria bernoulliana di parametro p .

I parametri di queste tre variabili aleatorie possono essere ricavati come segue:

$$\begin{aligned}\mu_1 &= \mu_X 2 p \\ \mu_2 &= \mu_X 2 (1 - p)\end{aligned}$$

Quest'ultime possono essere verificate per sostituzione nella formula:

$$\begin{aligned}E[X] &= \frac{1}{\mu_1} p + \frac{1}{\mu_2} (1 - p) \\ &= \frac{1}{\mu_X 2 p} p + \frac{1}{\mu_X 2 (1 - p)} (1 - p) \\ &= \frac{1}{2 \mu_X} + \frac{1}{2 \mu_X} \\ &= \frac{1}{\mu_X}\end{aligned}$$

Inoltre, si può ricavare il parametro p della bernoulliana a partire da:

$$\begin{aligned}
 C^2 &= \frac{1}{2p(1-p)} - 1 \\
 p(1-p) &= \frac{1}{2(C^2+1)} \\
 \Downarrow \quad p(1-p) &= \frac{1}{4}[1 - (2p-1)^2] \\
 1 - (2p-1) &= \frac{2}{C^2+1} \\
 2p-1 &= \sqrt{\frac{C^2-1}{C^2+1}} \\
 p &= \frac{1}{2} \left(1 + \sqrt{\frac{C^2+1}{C^2-1}} \right)
 \end{aligned}$$

4.5 Gli intervalli di confidenza

Per calcolare gli intervalli di confidenza viene utilizzata la seguente formula:

$$\left[\bar{X}_n - t_{\alpha/2, n-1} \frac{S_n}{\sqrt{n}}, \bar{X}_n + t_{\alpha/2, n-1} \frac{S_n}{\sqrt{n}} \right]$$

dove:

- $\bar{X}_n$: in generale è la media campionaria (calcolata su n campioni). I campioni in questo caso saranno le medie di ciascuna replica o batch a seconda del metodo utilizzato nello studio.
- S_n : è la varianza dei sudetti campioni.
- $t_{\alpha/2, n-1}$ è il valore critico della coda superiore di livello $\frac{\alpha}{2}$ di una variabile aleatoria di Student con $n-1$ gradi di libertà.

5 Modello computazionale

La simulazione di tipo next-event è stata implementata utilizzando Java come linguaggio di programmazione.

5.1 La gestione degli eventi

Gli eventi nel sistema sono stati modellati attraverso la classe astratta riportata in figura 2. L'attributo `eventType` rappresenta il tipo di evento che si vuole processare. Invece, l'attributo `timestamp` rappresenta il tempo in cui l'evento si è verificato. I tipi di eventi possibili sono:

- Arrivo
- Completamento

```
@Getter
public abstract class Event {
    private final double timestamp;
    private final EventType eventType;

    protected Event(double timestamp, EventType eventType) {
        this.timestamp = timestamp;
        this.eventType = eventType;
    }

    public abstract void process(SystemState s, EventVisitor visitor) {
        throws IllegalLifeException;
    }
}
```

Figura 2. Classe astratta che modella gli eventi nel sistema.

5.1.1 Evento di arrivo

Come si può osservare in figura 3, quando si verifica un evento di arrivo:

1. Si recupera il timestamp corrente e quello in cui si è verificato l'evento.
2. Viene calcolato l'avanzamento dei job specificando che non si tratta di un completamento.
3. Il job arrivato viene assegnato ad un server.
4. Si calcola il tempo del prossimo completamento: cambia a causa della politica di tipo processor sharing.
5. Se la simulazione non è terminata, viene generato il prossimo arrivo.
6. Si aggiorna il clock della simulazione.

La condizione di terminazione della simulazione viene impostata in base al tipo di studio che si vuole condurre:

- **Studio ad orizzonte finito:** la simulazione termina quando viene raggiunto un tempo limite.
- **Studio ad orizzonte infinito:** la condizione termina quando sono stati riempiti il numero di batch desiderati.

```
public class EventProcessor implements EventVisitor{
    @Override
    public void visit(SystemState s, ArrivalEvent event)
        throws IllegalLifeException {

        /* Get the servers in the infrastructure */
        Infrastructure infrastructure = s.getInfrastructure();

        /* Get the current clock and the one of this arrival */
        double startTs = s.getCurrent();
        double endTs = event.getTimestamp();

        infrastructure.computeJobsAdvancement(startTs, endTs, false);

        /* Add the next job to the list */
        double nextServiceLife = s.getServicesVA().gen();
        Job newJob = new Job(endTs, nextServiceLife);
        infrastructure.assignJob(newJob);

        /* Compute next completion time */
        double nextCompletionTs = infrastructure.computeNextCompletionTs(endTs);

        Event nextCompletion = new CompletionEvent(nextCompletionTs);
        s.addEvent(nextCompletion);

        /* Generate next arrival if simulation is not finished */
        boolean continueSimulating = REPETITION_NUMBER != 1 ?
            endTs < STOP :
            s.getAnalyzer().continueSimulating();

        if (continueSimulating) {
            double nextArrivalTs = endTs + s.getArrivalVA().gen();
            Event nextArrival = new ArrivalEvent(nextArrivalTs);
            s.addEvent(nextArrival);
        } else {
            s.addEvent(new ArrivalEvent(INFINITY));
        }

        /* Update the current system clock */
        s.setCurrent(endTs);
    }
}
```

Figura 3. Metodo per la gestione degli eventi di arrivo

5.1.2 Evento di completamento

Come si può osservare in figura 4, quando si verifica un evento di completamento:

1. Si recupera il timestamp corrente e quello in cui si è verificato l'evento.

2. Viene calcolato l'avanzamento dei job specificando che si tratta di un completamento.
3. Si calcola il tempo del prossimo completamento: cambia a causa dello scheduling di tipo processor sharing.
4. Si aggiorna il clock della simulazione.

```

@Override
public void visit(SystemState s, CompletionEvent event)
    throws IllegalLifeException {

    /* Get the servers in the infrastructure */
    Infrastructure infrastructure = s.getInfrastructure();

    /* Get the current clock and the one of this arrival */
    double startTs = s.getCurrent();
    double endTs = event.getTimestamp();

    /* Compute job advancement */
    infrastructure.computeJobsAdvancement(startTs, endTs, true);

    /* Plan scaling */
    planScaling(s, endTs, scalingIndicator);

    /* Generate next completion */
    double nextCompletionTs = infrastructure.activeJobExists() ?
        infrastructure.computeNextCompletionTs(endTs) :
        INFINITY;

    Event nextCompletion = new CompletionEvent(nextCompletionTs);
    s.addEvent(nextCompletion);

    /* Update the current system clock */
    s.setCurrent(endTs);
}

```

Figura 4. Metodo per la gestione degli eventi di completamento

5.2 Lo stato del sistema: la classe SystemState

Lo stato del sistema è mantenuto nella classe `SystemState`, nella quale troviamo:

- `current`: un `double` che rappresenta il clock della simulazione.
- `calendar`: una struttura dati per implementare la next-event simulation.
- `infrastructure`: una classe che rappresenta l'infrastruttura dei server.
- `serviceVA` e `arrivalsVA`: variabili aleatorie per generare i tempi dei prossimi eventi.
- `analyzer`: istanza della classe `Analyzer` che si occupa di analizzare le statistiche raccolte

durante la simulazione.

5.2.1 Gestione della coda degli eventi: la classe EventCalendar

Come richiesto dalla next-event simulation, viene mantenuto il riferimento solo al prossimo evento per ogni tipo. Questo comportamento viene modellato dalla classe `Calendar` ed in particolare dal metodo `nextEvent()` che ritorna l'evento con timestamp minore, come riportato nella figura 5.

```
public class EventCalendar {
    private final Map<EventType, Event> events = new HashMap<>();

    /**
     * Adds event in the map
     * @param e The event to add
     */
    public void addEvent(Event e) {
        events.put(e.getEventType(), e);
    }

    /**
     * Get event with minimum timestamp
     */
    public Event nextEvent() {
        return events.values()
            .stream()
            .min(Comparator.comparing(Event::getTimestamp))
            .orElse(null);
    }
}
```

Figura 5. Classe che modella la cosa degli eventi

5.2.2 Comportamento dell'infrastruttura del sistema: le classi Infrastructure

Per determinare come lo stato debba evolvere è stata definita un'interfaccia `Infrastructure` che astrae il comportamento del sistema. Essa può essere implementata senza lo spike server (utilizzando la classe `BaseServerInfrastructure`) oppure aggiungendo le funzionalità dello spike server (tramite la classe `SpikeServerDecorator` implementata mediante il pattern decorator). Questo permette di studiare i due comportamenti separatamente. Ogni classe di tipo `Infrastructure` mantiene gli indicatori per il routing dinamico e la lista di server a cui assegnare i job.

5.2.3 Astrazione dei server: le classi Server

Si usa la classe `AbstractServer` per definire tutte le componenti in comune tra i server, ovvero la propria capacità, la lista di job assegnati ad ognuno di essi e l'implementazione dell'avanzamento dei job con la politica processor sharing. La classe astratta viene poi estesa dalle due classi che modellano le due tipologie di Server possibili: `WebServer` e `SpikeServer`.

5.2.4 Astrazione dei job: le classi JobList e Job

Ogni server è collegato ad un'istanza della classe `JobList` che rappresenta l'insieme dei job assegnati ad esso. Un'istanza della classe `JobList` contiene una lista di job da gestire. Ciascun elemento

di questa lista è costituito da due attributi:

- `arrivalTime`: istante in cui il job è arrivato nel sistema.
- `remainingLife`: tempo d'esecuzione rimanente per il job.

La life di un job verrà decrementata, in funzione del tempo trascorso, durante il calcolo dell'avanzamento dei job.

5.3 Avanzamento dei job

Come si può vedere in figura 6, lo stesso avanzamento dei job viene utilizzato sia per gli eventi di arrivo che di completamento. Se si tratta di un completamento, viene aggiunta la rimozione a monte dell'avanzamento dei job: questa operazione viene effettuata in anticipo per risolvere un problema di instabilità nella precisione numerica dei calcoli, il quale causava la presenza di job con una life negativa. Nel caso non si tratti di un completamento, ma di un arrivo, viene semplicemente calcolato l'avanzamento dei job su tutti i server in base al tempo trascorso.

Come si può vedere in figura 7, ogni server calcola l'avanzamento dei propri job, secondo la politica processor sharing, in maniera indipendente rispetto agli altri. Inoltre, ciascun server calcola anche le proprie statistiche indipendentemente dagli altri server, come si può osservare in figura 8.

```
@Override
public double computeJobsAdvancement(
    double startTs,
    double endTs,
    boolean isCompletion) throws IllegalLifeException {

    Integer completionServerIndex = null;
    Double completedJobResponseTime = null;
    Job completedJob;

    if (isCompletion) {
        completionServerIndex = getCompletingServerIndex();
        AbstractServer minServer = allServers.get(completionServerIndex);
        completedJob = minServer.getMinRemainingLifeJob();
        completedJobResponseTime = endTs - completedJob.getArrivalTime();
        minServer.removeJob(completedJob);
    }

    /* Compute the advancement of each job in each web Server */
    for (int currIndex = 0; currIndex < allServers.size(); currIndex++) {
        AbstractServer server = allServers.get(currIndex);
        server.computeJobsAdvancement(
            startTs, endTs,
            Objects.equals(currIndex, completionServerIndex) ?
                completedJobResponseTime :
                null);
    }

    this.updateScalingIndicator();
    return base.scalingIndicator;
}
```

Figura 6. Metodo che gestisce l'avanzamento dei job su tutta l'infrastruttura

```

@Override
public void computeJobsAdvancement(
    double startTs,
    double endTs,
    Double completedJobResponseTime) throws IllegalLifeException {

    int completedJob = completedJobResponseTime == null ? 0 : 1;
    int jobAdvanced = jobs.size() + completedJob;

    stats.updateServerStats(startTs, endTs, jobAdvanced,
        completedJobResponseTime, this.serverState);

    // Update moving window response time
    if(completedJob == 1) {
        if(this.movingWindowResponseTime.size() == SLIDING_WINDOW_SIZE) {
            this.movingWindowResponseTime.removeFirst();
        }
        this.movingWindowResponseTime.addLast(completedJobResponseTime);
    }

    double quantum = (this.capacity / jobAdvanced) * (endTs - startTs);

    try{
        for (Job job : this.jobs.getJobs()) {
            job.decreaseRemainingLife(quantum);
        }
    } catch (IllegalLifeException e) {
        System.out.printf("endTs: \t%f", endTs);
        throw e;
    }
}

```

Figura 7. Metodo che gestisce l'avanzamento dei job su un server

```

public void updateServerStats(
    double startTs,
    double endTs,
    double jobNum,
    Double completedJobResponseTime,
    ServerState serverState) {

    assert startTs >= 0 && endTs >= startTs && jobNum >= 0;

    if (startTs < START || endTs < START) return;

    if(jobNum > 0) {
        this.nodeSum    += (endTs - startTs) * jobNum;
        this.serviceSum += (endTs - startTs);
    }

    /* Check if there has been a completion */
    boolean isCompletion = completedJobResponseTime != null;
    if (isCompletion) {
        this.completedJobs++;
        currMeanResponseTime +=
            (completedJobResponseTime - currMeanResponseTime) /
            completedJobs;
    }
}

```

Figura 8. Metodo che aggiorna le statistiche di un server

5.4 Le variabili aleatorie

Alla base delle implementazioni delle variabili aleatorie troviamo la classe **Rngs** progettata per la generazione di numeri pseudo-casuali multi-stream, in maniera più robusta rispetto alle librerie offerte da Java. Tale classe utilizza un **generatore di Lehmer** che con il multi-stream permette di generare dei numeri pseudo-random indipendenti. Nel modello base questa libreria è stata utilizzata per modellare le richieste dei job ed i tempi di arrivo utilizzando la distribuzione iperesponenziale.

5.4.1 Gestione dei seed

Lo stato di ciascuno stream è impostato attraverso la chiamata

```
rngs.plantSeed(int x)
```

dove **x** è il valore iniziale del seed. In particolare, attraverso questa funzione è possibile andare a settare lo stato di tutti gli stream “piantando” una sequenza di stati (seed).

5.4.2 La variabile aleatoria iperesponenziale

Per implementare la variabile aleatoria iperesponenziale è stato necessario utilizzare:

- Due variabili aleatorie esponenziali con parametri diversi.
- Una variabile aleatoria di Bernoulli: utilizzata per decidere quale ramo di esponenziale scegliere in ciascun campionamento.

Esse utilizzano tutte degli stream diversi in modo da usare generatori indipendenti. In particolare, per generare:

- I tempi di arrivo si utilizzano gli stream 0, 1 e 2

- Le domande dei job si utilizzano gli stream 3, 4 e 5.

5.5 La configurazione del sistema e gli esperimenti

Attraverso un file di configurazione `config.properties` è possibile scegliere i parametri del sistema, tra i quali ci sono: i parametri delle variabili aleatorie, i seed, il numero di server nel sistema ecc. Attraverso la classe `Config` è stato implementato un gestore centralizzato delle configurazioni del sistema che permette di caricare i parametri dal file di configurazione. La classe `Config` viene estesa da alcune sottoclassi che sovrascrivono alcuni dei parametri in base all'esperimento che si vuole eseguire.

5.6 Raccolta di dati e logging

La raccolta dei dati avviene attraverso delle classi di tipo `Collector`, un'interfaccia che espone i metodi:

- `collect()`: memorizza le statistiche per poter calcolare gli intervalli di confidenza.
- `analyzeAndPush()`: analizza i dati raccolti fino a quel momento e li riporta su un file CSV.

L'interfaccia `Collector` ha tre implementazioni:

- **ReplicationCollector**: utilizzata per studi ad orizzonte finito, raccoglie tutti i dati di una replica. Quando la replica è terminata, viene invocato il calcolo della media per le statistiche raccolte e i dati vengono riportati su un file CSV.
- **BatchMeanCollector**: raccoglie dati finché il batch non raggiunge il numero di completamenti richiesti per determinarne il riempimento. Una volta che il batch è completo viene calcolata la media delle statistiche al suo interno e i dati vengono riportati su un file CSV.
- **ServersTemporalBatchMeanCollector**: mentre la classe `BatchMeanCollector` costruisce i batch delle statistiche di sistema, questa classe suddivide in batch le statistiche dei server. La suddivisione, in questo caso, è basata sul tempo trascorso dall'apertura del batch, perché ciascun server gestisce un numero diverso di job.

5.7 Calcolo degli intervalli di confidenza

Come spiegato nella precedente sezione, le classi di tipo `Collector` calcolano le medie al completamento di ogni batch o di ogni replica. Questo viene fatto attraverso una classe di tipo `Analyzer` la cui interfaccia espone:

- Dei metodi per calcolare le medie dei dati raccolti fino al completamento della replica o del batch (quelli utilizzati dai `Collector`).
- I metodi per calcolare gli intervalli di confidenza che calcolano:
 1. La media delle medie dei batch/repliche.
 2. La varianza delle medie dei batch/repliche.
 3. Il valore critico della coda superiore di livello $\frac{\alpha}{2}$ di una variabile aleatoria di Student con $n - 1$ gradi di libertà.

Il calcolo avviene quando:

- Si raggiunge il numero di batch desiderati, nel caso del metodo batch mean.
- Si raggiungere il numero di repliche desiderate, nel caso del metodo delle repliche.

6 Verifica

Per accertarsi di aver costruito il sistema nel modo giusto si vuole rispondere alle seguenti domande:

1. Se il sistema si svuota, il numero di completamenti è uguale al numero di arrivi?
2. Il clock è strettamente crescente?
3. I risultati ottenuti dal nostro sistema sono coerenti con i risultati analitici?
4. La definizione delle variabili usate nella generazione dei numeri random è corretta?

6.1 Domanda 1

Tra i parametri di configurazione ce n'è uno, `EMPTY_JOBS`, che se settato a `true` fa terminare la simulazione quando il sistema ha terminato tutti i job. Quindi ci sarà un tempo massimo entro il quale si ricevono dei job. Dopo tale periodo, il sistema non riceve più richieste e tende a svuotarsi. Quando `EMPTY_JOBS = true` si attiva un'asserzione che controlla che il numero di job completati è pari al numero di job arrivati quando il sistema si è svuotato completamente.

6.2 Domanda 2

Il fatto che il clock sia monotono crescente è stato controllato tramite un `assert`.

6.3 Domanda 3

Per poter effettuare un confronto tra i risultati del modello simulativo e quello analitico, sono necessarie alcune semplificazioni. Questo perché la presenza del routing dinamico rende ciascun server un centro $G/H2/1/PS$, e quindi i risultati teorici affrontati durante il corso non sono utilizzabili. Motivo per cui si semplifica il sistema configurandolo come segue:

- Seed pari a 42.
- Lo spike server è spento.
- 1 web server di capacità $1 \frac{op.}{s}$.
- Batch mean con 16 batch di taglia 32768.

Si vuole simulare un centro $M/G/1/PS$ del quale sono noti il tasso di arrivo λ e il tasso di servizio μ . Di seguito sono riportate le formule utilizzate per calcolare i valori teorici in tabella 2.

$$\rho = \frac{\lambda}{\mu}$$
$$E[T_S] = \frac{1}{\mu - \lambda}$$

$$E[N_S] = \lambda E[T_S]$$

λ	ρ	$E[T_S](s)$	$E[N_S]$
0.6	0.6	2.5	1.5
0.8	0.8	5	4

Tabella 2. Valori teoria di un sistema $M/G/1/PS$

I risultati ottenuti, riportati in tabella 3, sono perfettamente in linea con quelli teorici riportati in tabella 2.

Distr(μ)	λ	ρ	$E[T_S](s)$	$E[N_S]$
exp	0.6	$0,600677 \pm 0,002783$	$2,502358 \pm 0,022781$	$1,504666 \pm 0,015297$
exp	0.8	$0,800896 \pm 0,003709$	$5,021040 \pm 0,112662$	$4,025796 \pm 0,095255$
h2	0.6	$0,601145 \pm 0,006454$	$2,498858 \pm 0,084586$	$1,502681 \pm 0,052216$
h2	0.8	$0,801529 \pm 0,008625$	$4,989004 \pm 0,341124$	$4,000316 \pm 0,274903$

Tabella 3. Risultati ottenuti con intervalli di confidenza al 95%

6.4 Domanda 4

In una simulazione, per asserirne la correttezza è fondamentale verificare il corretto funzionamento degli RNG utilizzati. La libreria `Rngs` è una libreria nota ed affidabile, quindi rimane da vedere se l'utilizzo degli strumenti che offre è corretto. A tale scopo è stato ideato un pacchetto di test per ciascuna distribuzione utilizzata. A titolo d'esempio, nelle figure 9 e 10 sono riportati i test fatti per la media e la varianza della variabile aleatoria iperesponenziale.

```
@Test
public void testH2Mean() {
    Rngs r = new Rngs();
    r.plantSeeds(SEED);

    // Hyperexponential creation
    HyperExponential ch2 =
        new HyperExponential(r, cv, mean, stream1, stream2, stream3);

    // Width of CI such that P[|X - E[X]| > eps] <= alpha
    double eps = Math.sqrt((Math.pow(mean, 2) * cv) / (n * alpha));

    // Sample Mean
    double sampleMean = computeSampleMean(ch2, n);

    ...

    // With probability (1 - alpha) this test pass
    assertTrue(mean >= sampleMean - eps && mean <= sampleMean + eps);
}
```

Figura 9. Test per la media della variabile aleatoria iperesponenziale

```

@Test
public void testH2Variance() {
    Rngs r = new Rngs();
    r.plantSeeds(SEED);

    // Hyperexponential creation
    HyperExponential ch2 =
        new HyperExponential(r, cv, mean, stream1, stream2, stream3);

    // Width of CI such that P[|X - E[X]| > eps] <= alpha
    double eps = Math.sqrt((Math.pow(mean, 2) * cv) / (n * alpha));

    double sampleMean = computeSampleMean(ch2, n);

    // Compute theoretical variance from definition of C^2
    double variance = cv * Math.pow(mean, 2);

    // With probability (1 - alpha) this test pass
    assertTrue(
        variance >= cv * Math.pow(sampleMean - eps, 2) &&
        variance <= cv * Math.pow(sampleMean + eps, 2)
    );
}

```

Figura 10. Test per la varianza della variabile aleatoria iperesponenziale

7 Validazione

Per accertarsi di aver costruito il sistema giusto si vuole rispondere alle domande:

1. Il sistema creato è coerente con i risultati ottenuti dallo studio originale?
2. Lo spike server effettivamente aiuta a decongestionare il sistema durante fluttuazioni short-term?
3. Aumentare/Diminuire il numero di server migliora/peggiora le performance?

7.1 Domanda 1

Per rispondere alla prima domanda configuriamo il sistema come segue:

- Seed pari a 42.
- Lo spike server è acceso con $SI^{\max} = 140$.
- Un web server con capacità $1 \frac{\text{op.}}{\text{sec.}}$.
- Tempi di interarrivo iperesponenziali di media 0.15 e coefficiente di variazione pari a 4.
- Domande dei job iperesponenziali di media 0.16 e coefficiente di variazione pari a 4.

Il risultato di questo esperimento è visibile in figura 12. Confrontando i risultati ottenuti con la figura 11 possiamo dire che i risultati ottenuti sono coerenti con quelli ottenuti nello studio originale.

7.2 Domanda 2

Per vedere che lo spike server aiuta nella decongestione basta osservare la figura 12. Da questa figura si vede che in corrispondenza dei picchi del numero di job nel web server, lo spike server entra in azione e smaltisce il carico al posto del web server.

7.3 Domanda 3

Per rispondere a questa domanda è stata utilizzata la seguente configurazione:

- Seed pari a 42.
- Lo spike server è spento.
- Batch mean con 16 batch di taglia 32768.
- Distribuzione degli arrivi esponenziale.
- Distribuzione del servizio iperesponenziale.

In questo esperimento viene fatto il confronto tra la simulazione di un sistema $M/H2/1/PS$ e di un sistema $M/H2/2/PS$. I risultati ottenuti sono stati riportati nella tabella 4 e permettono di dire che l'aggiunta di un server diminuisce il tempo di risposta che risulta più che dimezzato. Inoltre, diminuisce anche il numero di job presenti nel sistema. Questo risultato è perfettamente coerente con quello che ci si aspetta a livello teorico.

Num. WS	ρ	$E[T_S](s)$	$E[N_S]$
1	$0,601145 \pm 0,006454$	$2,498858 \pm 0,084586$	$1,502681 \pm 0,052216$
2	$0,601146 \pm 0,006462$	$1,153003 \pm 0,016681$	$0,693313 \pm 0,010719$

Tabella 4. Confronto tra la simulazione di un sistema $M/H2/1/PS$ e di un sistema $M/H2/2/PS$

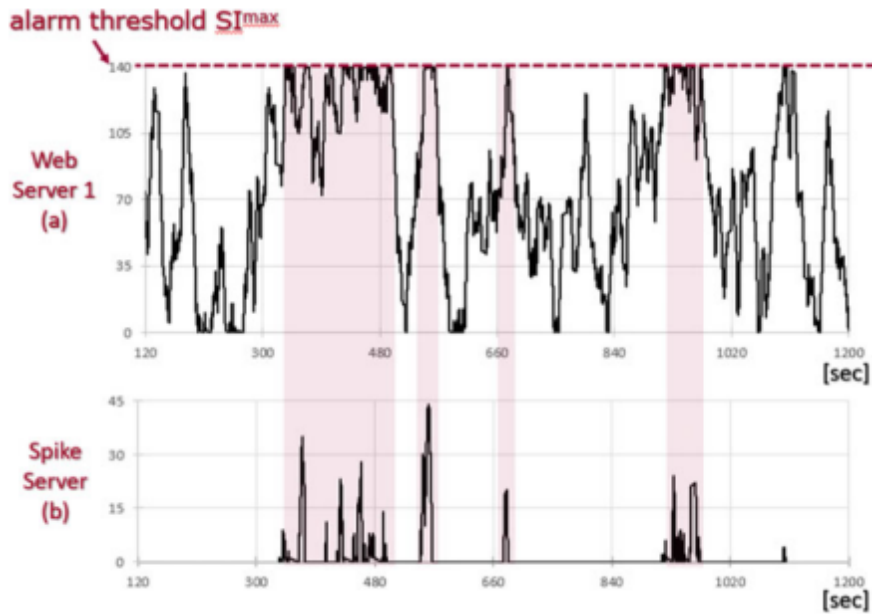


Figura 11. Risultati ottenuti nello studio originale del libro di testo

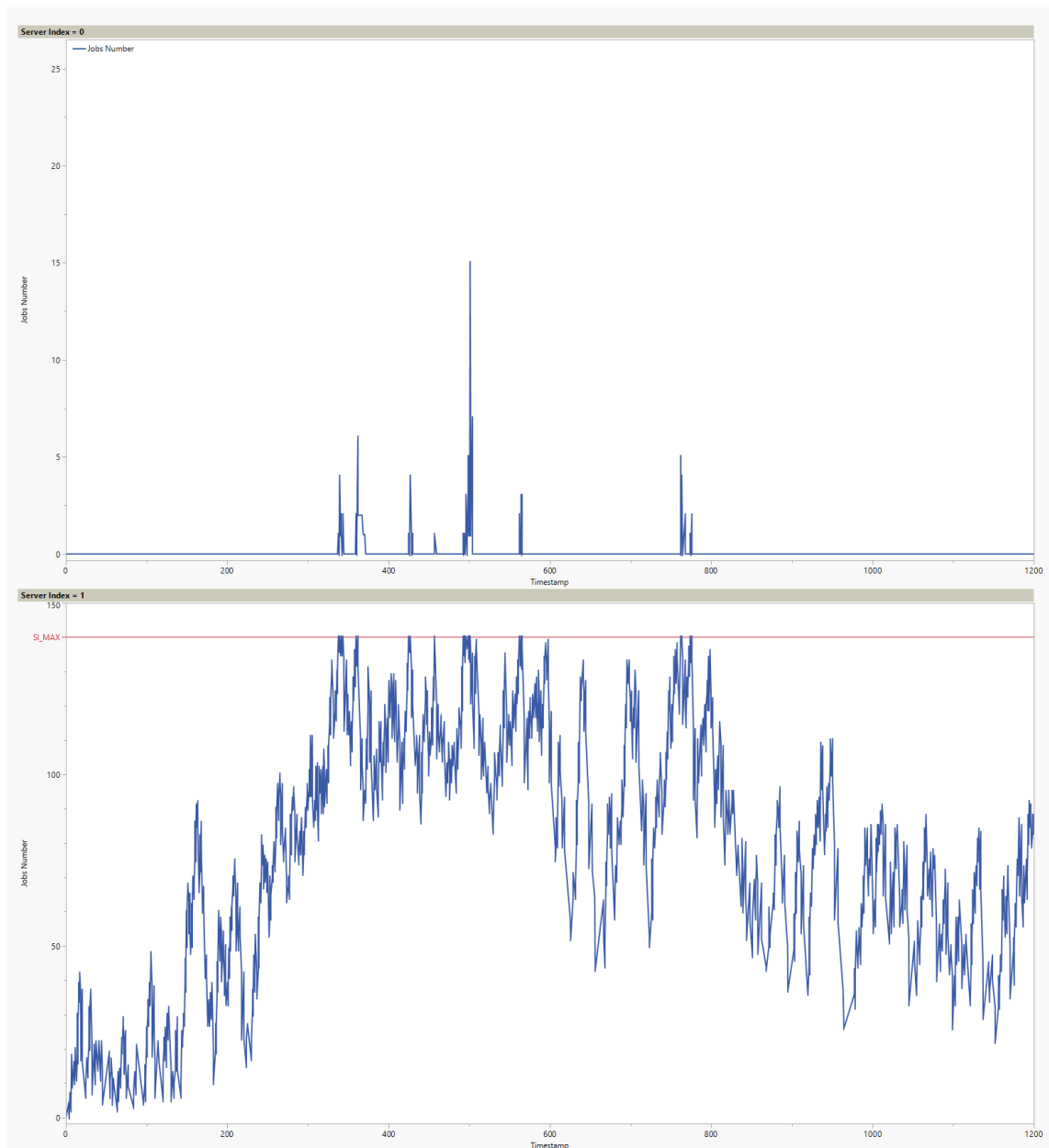


Figura 12. Risultato del primo esperimento di validazione

8 Studio del transitorio

In questa sezione viene riportato lo studio del transitorio per alcune configurazioni. Questo ci permette di assicurare che i parametri utilizzati negli esperimenti successivi corrispondano a delle simulazioni le cui statistiche convergono. I parametri, fissati in tutti gli studi, sono riportati nella tabella 5. Dato che gli obiettivi prevedono lo studio dello spike server nella gestione delle fluttuazioni short-term, nello studio del transitorio del modello base, sono disabilitate le fluttuazioni long term.

Distr(λ)	Distr(Z)	λ	$E[Z]$
h2(cv=4)	h2(cv=4)	$\frac{1}{0.15} \frac{\text{req}}{\text{sec.}}$	0.16 op.

Tabella 5. I parametri del sistema utilizzati nello studio del transitorio

8.1 Studio 1

In questo studio si mostra la convergenza delle statistiche di un sistema con $SI^{\max} = 10$. I risultati in forma grafica sono riportati nella figura 13. Invece, i risultati in forma tabellare, ottenuti con 32 repliche e intervalli di confidenza al 95% utilizzando la distribuzione di Student sono riportati in tabella 6.

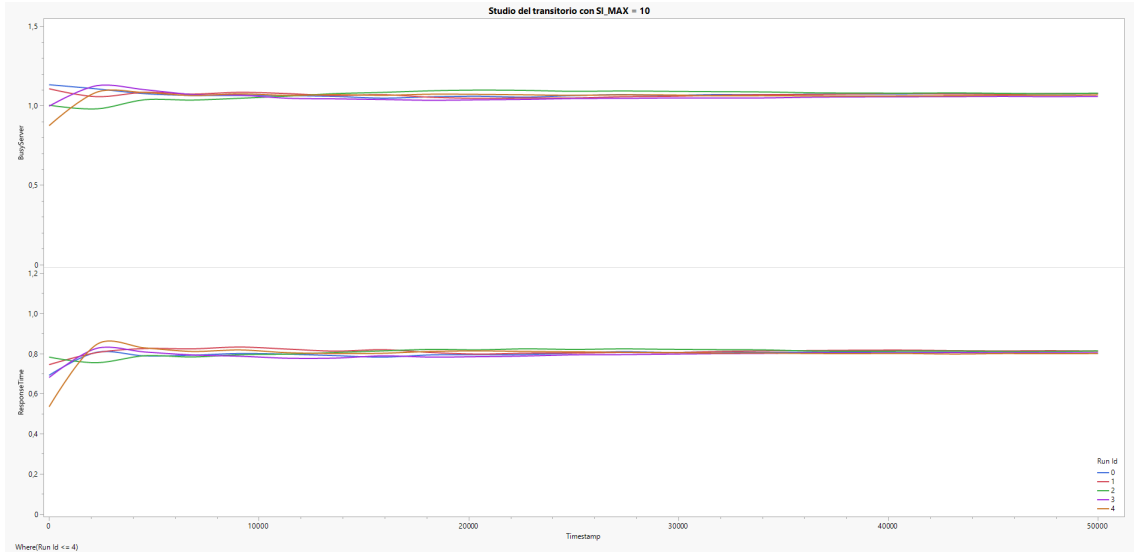


Figura 13. Studio del transitorio del modello base con $SI^{\max} = 10$

Tempo di risposta	Utilizzazione
$0,801229 \pm 0,003391$	$1,066962 \pm 0,003763$

Tabella 6. Intervalli di confidenza al 95% calcolati con 32 ripetizioni e la distribuzione di Student

Id della config.	Stream					
	0	1	2	3	4	5
0	42	962850	598499780	334435817	424484935	1068730318
1	818259835	1035305747	677766728	57986418	1539250494	1378985233
2	1190930560	1957535807	642181558	1168727680	1888195867	1265716530
3	1296326881	1728877251	2104862923	1087043410	1759238428	1847488060
4	189209668	1595262914	2084343913	180649430	1891825369	330488834

Tabella 7. Seed iniziali degli stream per ciascuna run

8.2 Studio 2

In questo studio si mostra la convergenza delle statistiche di un sistema con $SI^{\max} = 80$. I risultati in forma grafica sono riportati nella figura 14. Invece, i risultati in forma tabellare, ottenuti con 32 repliche e intervalli di confidenza al 95% utilizzando la distribuzione di Student sono riportati in tabella 8.

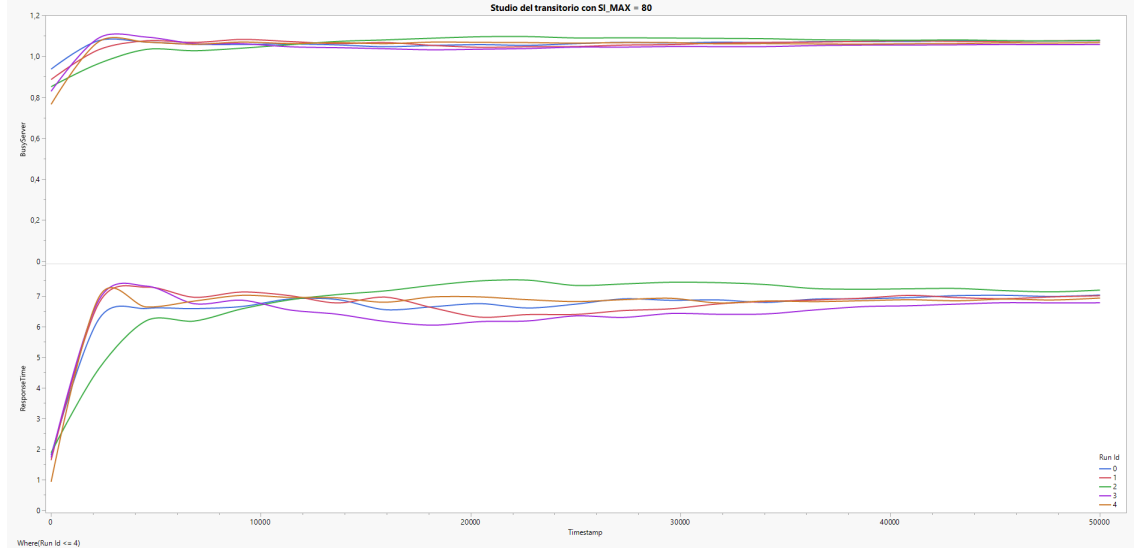


Figura 14. Studio del transitorio del modello base con $SI^{\max} = 80$

Tempo di risposta	Utilizzazione
$6,888811 \pm 0,089487$	$1,065896 \pm 0,003739$

Tabella 8. Intervalli di confidenza al 95% calcolati con 32 ripetizioni e la distribuzione di Student

Id della config.	Stream					
	0	1	2	3	4	5
0	42	962850	598499780	334435817	424484935	1068730318
1	818259835	1035305747	677766728	1820675483	1539250494	1533042986
2	1190930560	1957535807	642181558	2116677626	1888195867	1394371324
3	1296326881	1728877251	2104862923	647081215	1759238428	1055932503
4	189209668	1595262914	2084343913	2044877830	1891825369	1595088293

Tabella 9. Seed iniziali degli stream per ciascuna run

8.3 Studio 3

In questo studio si mostra la convergenza delle statistiche di un sistema con $SI^{\max} = 160$. I risultati in forma grafica sono riportati nella figura 15. Invece, i risultati in forma tabellare, ottenuti con 32 repliche e intervalli di confidenza al 95% utilizzando la distribuzione di Student sono riportati in tabella 10.

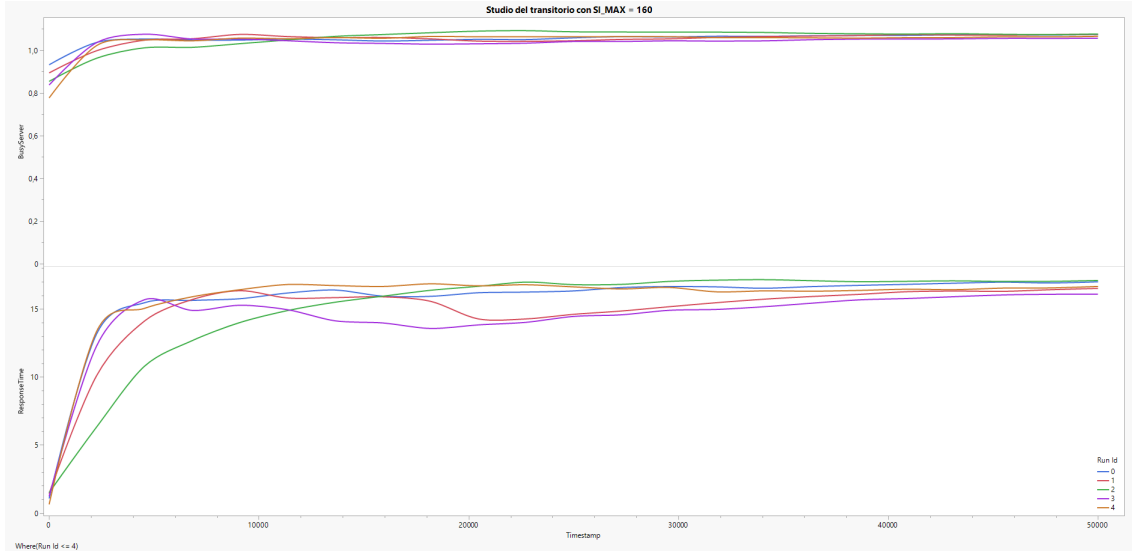


Figura 15. Studio del transitorio del modello base con $SI^{\max} = 160$

Tempo di risposta	Utilizzazione
$16,362239 \pm 0,260909$	$1,064104 \pm 0,003696$

Tabella 10. Intervalli di confidenza al 95% calcolati con 32 ripetizioni e la distribuzione di Student

Id della config.	Stream					
	0	1	2	3	4	5
0	42	962850	598499780	334435817	424484935	1068730318
1	818259835	1035305747	677766728	57986418	1539250494	1378985233
2	1190930560	1957535807	642181558	1168727680	1888195867	1265716530
3	1296326881	1728877251	2104862923	107683650	1759238428	393490768
4	189209668	1595262914	2084343913	2044877830	1891825369	1595088293

Tabella 11. Seed iniziali degli stream per ciascuna run

9 Esperimenti

In questa sezione vengono descritti gli esperimenti ad orizzonte infinito. La scelta dell'orizzonte deriva dal fatto che per il modello base si vuole studiare il comportamento dello spike server nella gestione delle fluttuazioni short-term. Tali fluttuazioni sono presenti per tutta la durata della simulazione e sono dovute alla variabilità della distribuzione iperesponenziale.

9.1 Esperimento 1

La configurazione utilizzata prevede:

- Tempo di interarrivo iperesponenziale di media 0.15 e coefficiente di variazione 4.
- Richieste dei job iperesponenziali di media 0.16 e coefficiente di variazione 4.
- I server hanno tutti capacità $1 \frac{\text{op.}}{s}$.
- Batch mean con 16 batch di dimensione 32768.

Per sintonizzare i parametri dell'autoscaler valutiamo gli effetti della soglia di allarme $SI^{\max}$ sul tempo di risposta del sistema. In particolare, è stata condotta un'analisi di tipo what-if con il parametro di controllo $SI^{\max}$ che va da 10 a 160 con incrementi di 10. Come si può osservare nelle figure 16 e 17, quando $SI^{\max}$ aumenta, anche il throughput ed il tempo di risposta per il web server aumentano. Gli stessi indici per lo spike server decrescono all'aumentare di $SI^{\max}$, come si vede nelle figure 18 e 19. Gli stessi risultati in forma numerica si possono osservare nelle tabelle 12 e 13. E' importante notare che stiamo valutando il comportamento di questi indici di prestazione mantenendo fisso il tasso di arrivo. Dalla figura 20 si può osservare che il valore di $SI^{\max}$ che rende il tempo di risposta del sistema il più vicino possibile ad 8s è $SI^{\max} = 90$. Gli stessi risultati, sotto forma di tabella, sono riportati in tabella 14.

$SI^{\max}$	Throughput	Response Time
10	5,127875 $\pm$ 0,033106	0,909282 $\pm$ 0,010984
20	5,651191 $\pm$ 0,031067	1,712261 $\pm$ 0,036961
30	5,879554 $\pm$ 0,027472	2,543374 $\pm$ 0,064656
40	5,998077 $\pm$ 0,025535	3,425360 $\pm$ 0,123392
50	6,072682 $\pm$ 0,037480	4,376537 $\pm$ 0,189155
60	6,125793 $\pm$ 0,031226	5,370719 $\pm$ 0,247300
70	6,160074 $\pm$ 0,040679	6,416857 $\pm$ 0,330334
80	6,172627 $\pm$ 0,040030	7,596067 $\pm$ 0,443125
90	6,181249 $\pm$ 0,043808	8,815972 $\pm$ 0,550488
100	6,199519 $\pm$ 0,043950	10,018892 $\pm$ 0,643670
110	6,199816 $\pm$ 0,043632	11,341997 $\pm$ 0,721791
120	6,211839 $\pm$ 0,052712	12,568786 $\pm$ 0,881524
130	6,217687 $\pm$ 0,048009	13,925087 $\pm$ 0,965177
140	6,219765 $\pm$ 0,052420	15,314448 $\pm$ 1,003254
150	6,222038 $\pm$ 0,055231	16,673021 $\pm$ 1,141938
160	6,223292 $\pm$ 0,051550	18,135398 $\pm$ 1,217655

Tabella 12. Intervalli di confidenza per gli indici del Web Server

$SI^{\max}$	Throughput	Response Time
10	$1,556932 \pm 0,035753$	$0,474091 \pm 0,026323$
20	$1,033898 \pm 0,049119$	$0,434404 \pm 0,027829$
30	$0,805181 \pm 0,046198$	$0,421452 \pm 0,019059$
40	$0,686596 \pm 0,059670$	$0,414522 \pm 0,019751$
50	$0,611958 \pm 0,068470$	$0,419926 \pm 0,018367$
60	$0,558818 \pm 0,065273$	$0,421549 \pm 0,020454$
70	$0,524513 \pm 0,070921$	$0,420493 \pm 0,015962$
80	$0,511941 \pm 0,077895$	$0,419453 \pm 0,012926$
90	$0,503262 \pm 0,084095$	$0,414740 \pm 0,013115$
100	$0,484921 \pm 0,085512$	$0,418590 \pm 0,018802$
110	$0,484591 \pm 0,087837$	$0,415307 \pm 0,017220$
120	$0,472541 \pm 0,092829$	$0,418174 \pm 0,019497$
130	$0,466696 \pm 0,090373$	$0,424537 \pm 0,022485$
140	$0,464521 \pm 0,094070$	$0,423399 \pm 0,013123$
150	$0,462206 \pm 0,099322$	$0,430317 \pm 0,015189$
160	$0,460934 \pm 0,095729$	$0,433150 \pm 0,013109$

Tabella 13. Intervalli di confidenza per gli indici dello Spike Server

$SI^{\max}$	Response Time
10	$0,803330 \pm 0,013989$
20	$1,511701 \pm 0,030680$
30	$2,278816 \pm 0,061184$
40	$3,085543 \pm 0,108611$
50	$3,951199 \pm 0,162778$
60	$4,913509 \pm 0,225489$
70	$5,826734 \pm 0,320373$
80	$6,940984 \pm 0,404713$
90	$8,036149 \pm 0,466445$
100	$9,168558 \pm 0,544738$
110	$10,382310 \pm 0,637147$
120	$11,500710 \pm 0,773449$
130	$12,758796 \pm 0,869986$
140	$14,012848 \pm 0,890357$
150	$15,235599 \pm 1,023685$
160	$16,608294 \pm 1,100550$

Tabella 14. Tempo di risposta del sistema al variare di $SI^{\max}$

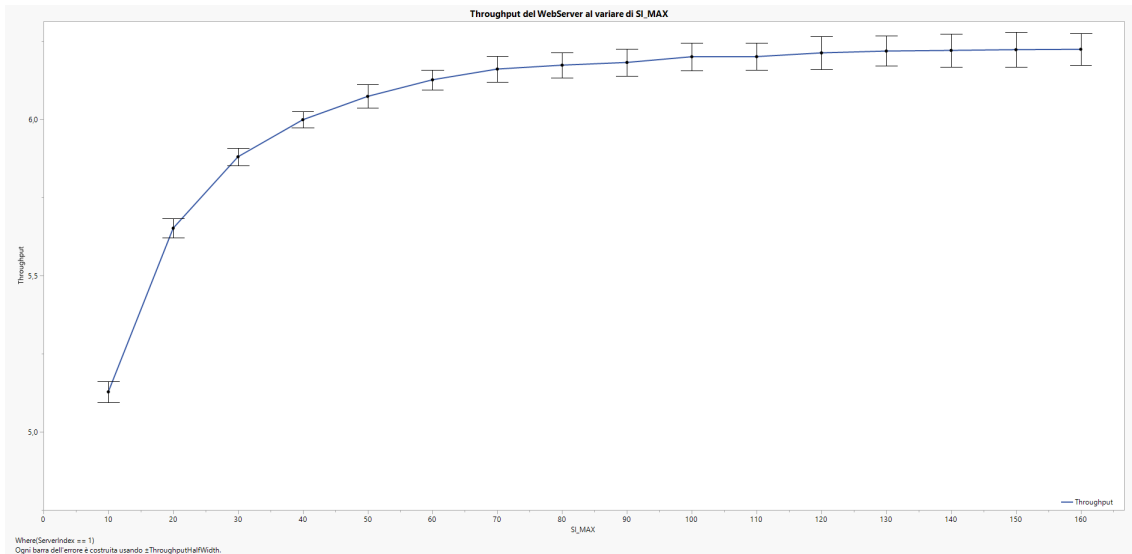


Figura 16. Throughput del web server al variare di SI^{max}

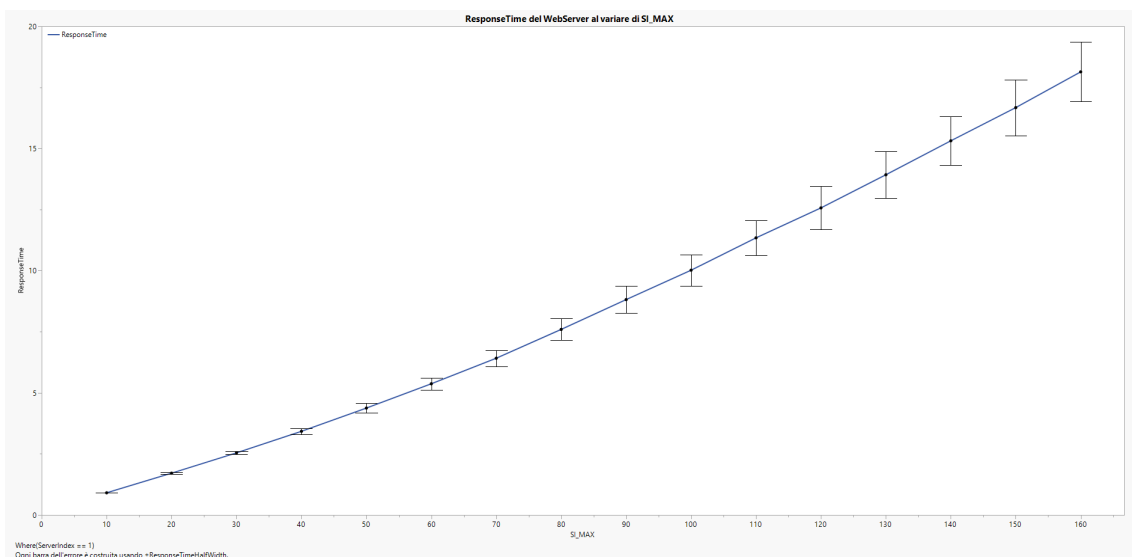


Figura 17. Tempo di risposta del web server al variare di SI^{max}

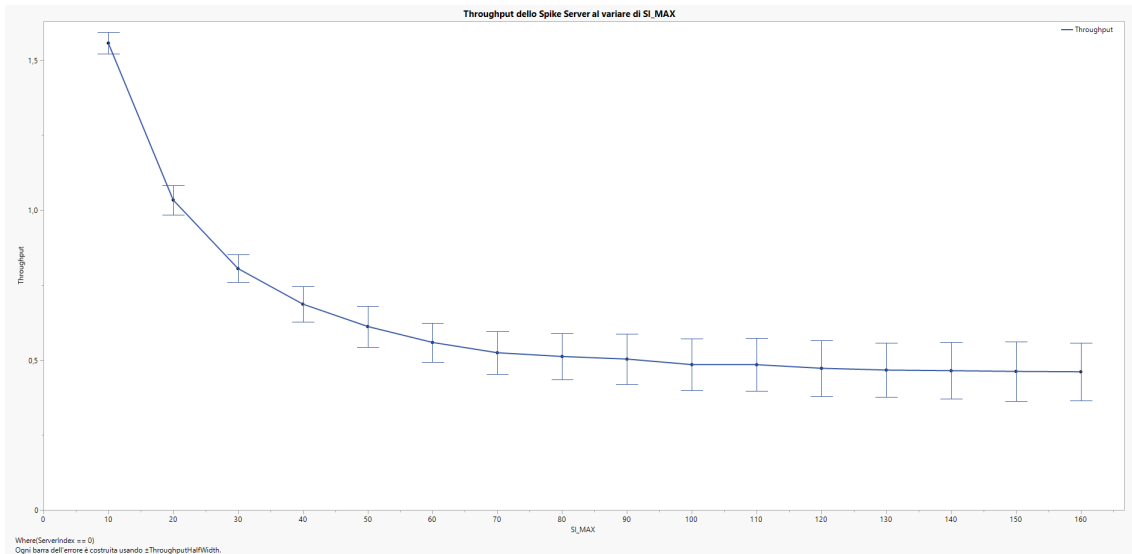


Figura 18. Throughput dello spike server al variare di $SI^{\max}$

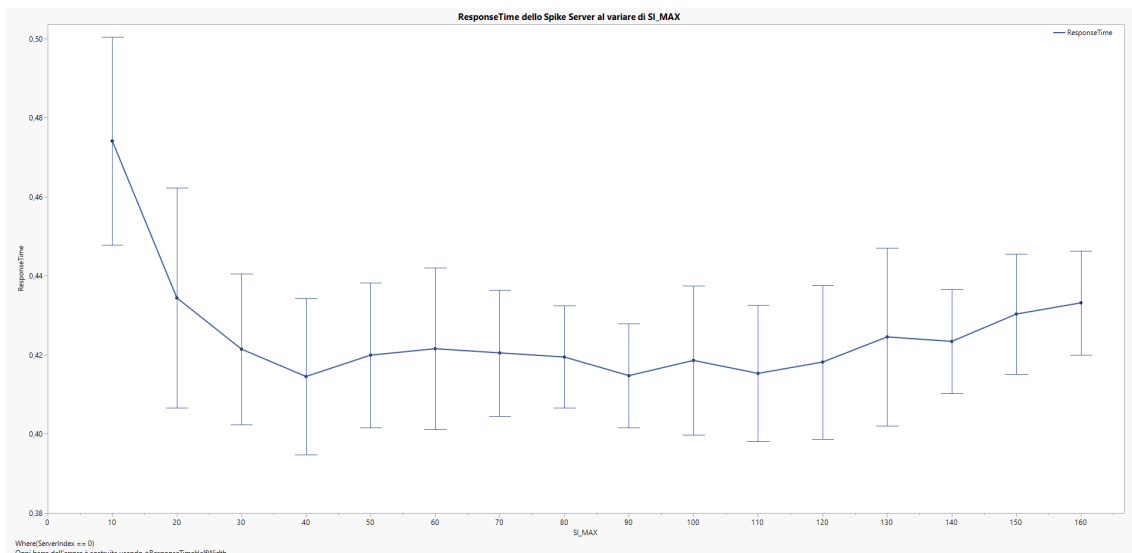


Figura 19. Tempo di risposta dello spike server al variare di $SI^{\max}$

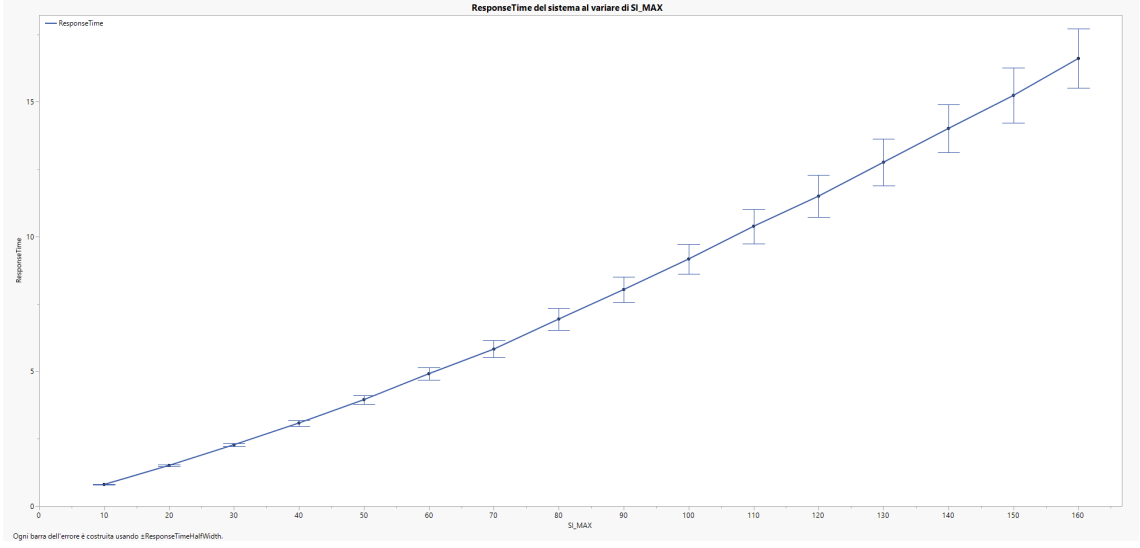


Figura 20. Tempo di risposta del sistema al variare di $SI^{\max}$

9.2 Esperimento 2

La configurazione utilizzata per questo studio prevede:

- Richieste dei job iperesponenziali di media 0.16 e coefficiente di variazione 4.
- I server hanno tutti capacità $1 \frac{op.}{s}$.
- Batch mean con 16 batch di dimensione 32768.

Questo esperimento consiste in una analisi di tipo what-if per valori di $SI^{\max}$ da 40 a 160 variando il tasso di arrivo da 1 a 12 richieste al secondo. L'impatto positivo del meccanismo di routing dinamico basato sullo stato è evidenziato dalla figura 21 ed in tabella 15. La curva inferiore rappresenta il tempo di risposta del sistema per $SI^{\max}=40$, mentre quella superiore per $SI^{\max}=160$. Si possono identificare 3 diverse configurazioni di carico:

- Basso: $1 \frac{req.}{s} \leq \lambda \leq 6 \frac{req.}{s}$
- Medio: $6 \frac{req.}{s} < \lambda \leq 10 \frac{req.}{s}$
- Alto: $10 \frac{req.}{s} < \lambda \leq 12 \frac{req.}{s}$

9.2.1 Analisi del sistema a basso carico

Si può osservare che il tempo di risposta del sistema è sempre inferiore ai 10s per qualsiasi valore di $SI^{\max}$. Inoltre, è importante notare che in assenza dello spike server, passare da $\lambda = 6 \frac{req.}{s}$ a $\lambda = 7 \frac{req.}{s}$ causa un aumento importante del tempo di risposta. Questo aumento è dovuto al fatto che il web server ha un'utilizzazione molto elevata e il throughput tende al suo limite teorico. Ricordiamo che il massimo tasso di arrivo che il web server può processare quando è saturo è dato da:

$$\lambda_{\max} = \frac{1}{D_{WS}}$$

$$= 6.25 \frac{\text{req.}}{s}$$

La precedente formula è stata ricavata dalla legge dell'utilizzazione che dice che:

$$U = \lambda D$$

9.2.2 Analisi del sistema con carico medio

Queste configurazioni mostrano valori crescenti del tempo di risposta del sistema fino al tasso di arrivo di circa $8 \frac{\text{req.}}{s}$. Questo aumento è dovuto principalmente all'aumento del tasso di arrivo al web server che è vicino alla congestione. Un ulteriore aumento del tasso di arrivo da $8 \frac{\text{req.}}{s}$ a $10 \frac{\text{req.}}{s}$ genera un aumento nella rivelazione della violazione della soglia $SI^{\max}$. Quindi lo spike server riceverà più richieste permettendo al web server di decongestionarsi abbassando così il tempo di risposta del sistema.

9.2.3 Analisi del sistema con carico alto

Queste configurazioni permettono di osservare la saturazione dello spike server che diventa sempre più evidente man mano che il valore di $SI^{\max}$ diminuisce tra 160 e 40. Questo perché l'aumento del tasso di arrivo rende più frequenti le violazioni della soglia $SI^{\max}$, il che fa aumentare l'utilizzazione dello spike server. Utilizzare soglie più basse rende ancora più frequente l'utilizzo dello spike server.

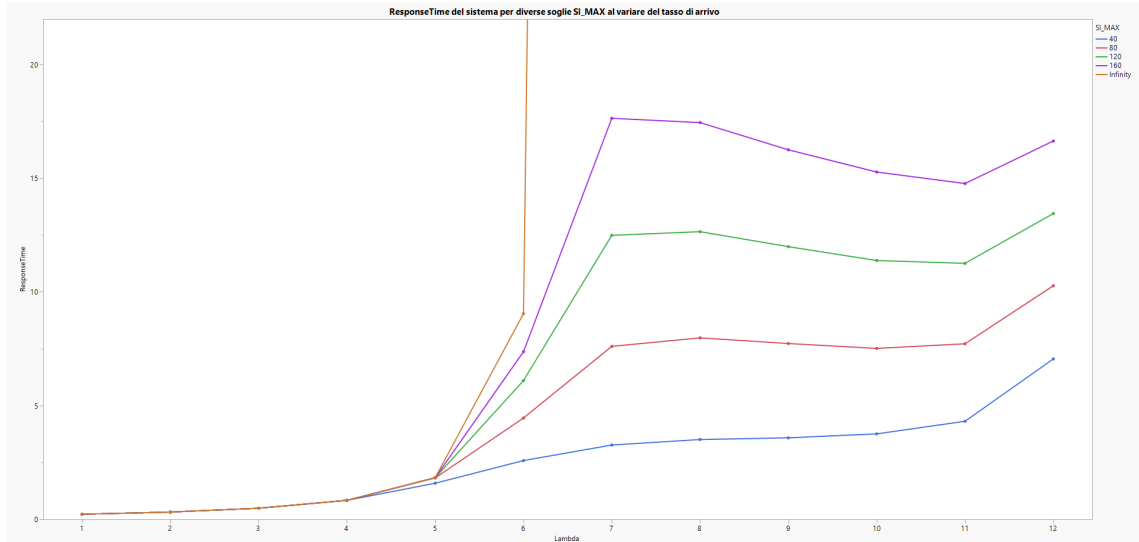


Figura 21. Tempo di risposta del sistema per diverse soglie $SI^{\max}$ al variare del tasso di arrivo

λ	$SI^{\max}$	Tempo di risposta
1	Qualsiasi	$0,219910 \pm 0,003463$
2	Qualsiasi	$0,315380 \pm 0,006851$
3	Qualsiasi	$0,484806 \pm 0,012058$
4	Infinito	$0,831213 \pm 0,036886$
	80, 120, 160	$0,831579 \pm 0,037064$
5	Infinito	$1,828404 \pm 0,235176$
	40	$1,584259 \pm 0,114479$
	80	$1,816041 \pm 0,226551$
	120, 160	$1,828404 \pm 0,235176$
6	Infinito	$9,043254 \pm 2,656283$
	40	$2,580790 \pm 0,121695$
	80	$4,454872 \pm 0,541404$
	120	$6,100170 \pm 1,043269$
	160	$7,370385 \pm 1,539838$
7	40	$3,263771 \pm 0,094520$
	80	$7,603163 \pm 0,287182$
	120	$12,486700 \pm 0,498775$
	160	$17,635027 \pm 0,722027$
8	40	$3,507288 \pm 0,069034$
	80	$7,977017 \pm 0,113147$
	120	$12,648019 \pm 0,186358$
	160	$17,444911 \pm 0,287433$
9	40	$3,580970 \pm 0,046116$
	80	$7,728228 \pm 0,089405$
	120	$11,989132 \pm 0,146951$
	160	$16,247787 \pm 0,255481$
10	40	$3,754897 \pm 0,118277$
	80	$7,514190 \pm 0,091973$
	120	$11,380003 \pm 0,150683$
	160	$15,272135 \pm 0,239219$
11	40	$4,309787 \pm 0,341493$
	80	$7,716563 \pm 0,281914$
	120	$11,255388 \pm 0,234225$
	160	$14,765593 \pm 0,312682$
12	40	$7,050733 \pm 1,325892$
	80	$10,269899 \pm 1,306929$
	120	$13,444894 \pm 1,381088$
	160	$16,635365 \pm 1,354873$

Tabella 15. Tempo di risposta del sistema configurato con diversi valori di $SI^{\max}$ e al variare del tasso di arrivo

10 Introduzione al modello migliorativo

Il modello base prevede un sistema costituito da un web server ed uno spike server. Lo spike server consente di gestire le situazioni in cui il carico aumenta per un breve periodo. In particolare, quando il web server supera una soglia, che indica la sua congestione, le richieste vengono inoltrate allo spike server. Questo permette al web server di decongestionarsi e quindi di poter ricevere di nuovo richieste dopo il periodo di congestione. Tuttavia, quando le fluttuazioni si propagano per un tempo maggiore, l'utilizzazione dello spike server aumenta non consentendo al web server di decongestionarsi. Questo causa un aumento del tempo di risposta del sistema, come si può osservare in figura 22. Questi risultati sono stati ottenuti con la stessa configurazione del esperimento in sezione 9.1 ma in presenza di fluttuazioni long-term. Confrontando le figure 20 e 22 si può osservare che in entrambi i casi il tempo di risposta del sistema cresce al variare di SI^{max} . Tuttavia, nella figura 22 il tempo di risposta varia tra circa 17 e 30 secondi. In assenza di fluttuazioni long term il tempo di risposta del sistema variava tra circa 0.8 e 16 secondi (figura 20). Per tale motivo è stato introdotto un meccanismo di scaling orizzontale che accende e spegne web server in base al carico effettivo. Questo non solo permette di migliorare le performance, grazie al numero maggiore di web server, ma anche di non sprecare risorse spegnendoli quando il carico diminuisce.

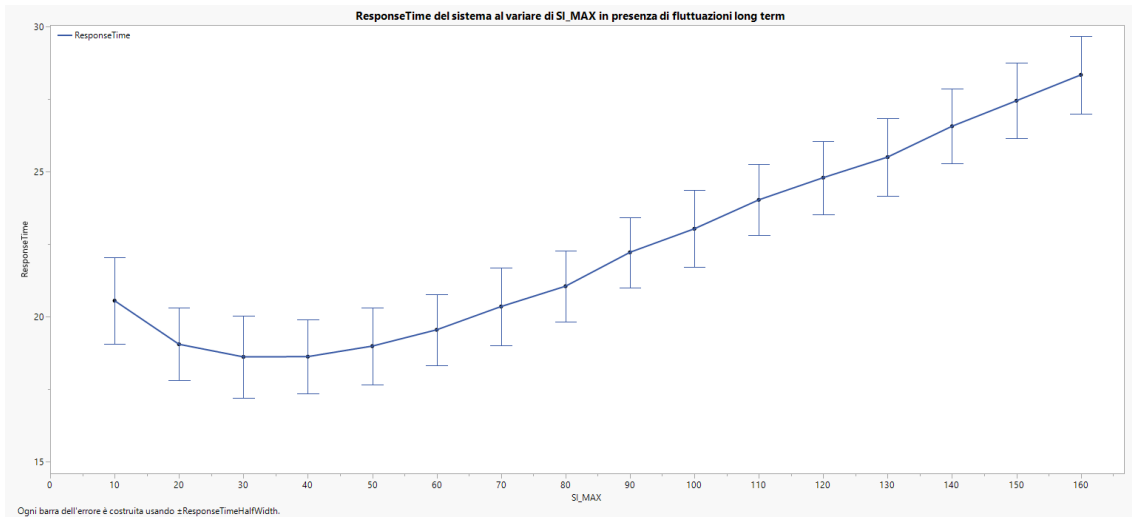


Figura 22. Tempo di risposta del sistema al variare di SI^{max} in presenza delle fluttuazioni long-term

11 Modello concettuale migliorativo

11.1 Componenti

11.1.1 I Web Server

La presenza del autoscaler orizzontale permette di accendere e spegnere i server sulla base di un indicatore di scaling. Tuttavia, l'accensione di un server non è un'operazione immediata. Infatti, un server sarà pronto a ricevere le richieste solamente dopo il tempo di booting. Allo stesso modo un server prima che verrà spento definitivamente deve aver completato tutti i job che aveva in carico quando è stato richiesto il suo spegnimento. Quindi è stato necessario aggiungere i seguenti stati:

- **ACTIVE:**
 - Il WS può essere scelto per servire una nuova richiesta.
 - Il WS può avere delle richieste in carico oppure no.
- **TO_BE_REMOVED:**
 - Il WS non può ricevere delle nuove richieste.
 - Il WS ha delle richieste in carico.
 - Portare il WS nello stato **ACTIVE** richiede un tempo trascurabile.
- **REMOVED:**
 - Il WS non può ricevere richieste.
 - Il WS non ha delle richieste in carico.
 - Portare il WS nello stato **ACTIVE** richiede un tempo **non** trascurabile.
- **TO_BE_ACTIVE:**
 - Il WS non può ricevere richieste.
 - Il WS non ha delle richieste in carico.
 - Il WS è in fase di attivazione.

11.1.2 Il load controller

Nel modello migliorativo il load controller viene esteso per prendere le decisioni di scaling. Per poter decidere quando aggiungere/rimuovere un server viene monitorato un indicatore di performance basato sul tempo di risposta o sul numero di job nel sistema.

11.2 Gli eventi del sistema

Lo scaling orizzontale viene modellato attraverso i seguenti eventi:

- **Richiesta di scaling out:** nel momento in cui il load controller decide che è necessaria un'operazione di scaling out effettua una richiesta. Il completamento di questo evento porterà il server scelto nello stato **TO_BE_ACTIVE**.

- **Scaling out:** operazione che materializza la richiesta di scaling out (dopo che è trascorso il tempo non trascurabile di accensione del server). Il completamento di questo evento porterà il server scelto nello stato **ACTIVE**.
- **Scaling in:** il server che deve essere spento verrà portato nello stato:
 - **TO_BE_REMOVED** se aveva dei job da gestire. Quando avrà servito tutte le richieste pendenti verrà portato nello stato **REMOVED**. A meno che, nel frattempo non viene fatta una richiesta di scaling out che lo riguarda. In tal caso, verrà riportato direttamente nello stato **ACTIVE**.
 - **REMOVED:** se non aveva job da gestire.

12 Modello delle specifiche migliorativo

12.1 Inizializzazione del sistema

All'inizio della simulazione è acceso un solo web server, che si troverà nello stato **ACTIVE**, e lo spike server. Tutti gli altri web server saranno nello stato **REMOVED**.

12.2 Modellazione dell'autoscaling orizzontale

12.2.1 Indicatori di scaling

Il load controller per prendere decisioni di scaling può monitorare due diversi indicatori di scaling:

- **Basato sul tempo di risposta dei web server:** per ciascun web server viene calcolata una media del tempo di risposta utilizzando una finestra scorrevole. Il valore di questo indicatore viene aggregato calcolando una media finale. L'aggiornamento di questo indicatore avviene ad ogni completamento.
- **Basato sul numero di job nel sistema:** l'indicatore viene calcolato come la somma dei job in carico presso i server negli stati **ACTIVE** e **TO_BE_REMOVED** e presso lo spike server. L'aggiornamento di questo indicatore avviene ad ogni arrivo e completamento.

12.2.2 Le decisioni di scaling

Per decidere se è necessaria una decisioni di scaling oppure no si utilizza il seguente approccio:

1. Nella configurazione della simulazione viene impostata una soglia che viene utilizzata in maniera diversa a seconda del tipo di indicatore scelto:

- **Indicatore basato sul tempo di risposta:**
 - Azione di scaling out intrapresa se l'indicatore è superiore al 150% del valore della soglia impostata.
 - Azione di scaling in intrapresa se l'indicatore è inferiore al 50% del valore della soglia impostata.
- **Indicatore basato sul numero di job nel sistema:** la soglia scelta in fase di configurazione viene utilizzata in questo modo:
 - Viene definita la **SCALING_IN_THR** come il 60% del valore della soglia scelta in fase di configurazione.
 - La soglia scelta in fase di configurazione è la soglia **SCALING_OUT_THR**.

I valori di queste due soglie vengono utilizzati per definire il numero di server che indicano la necessità di un'operazione di scaling in funzione del numero di job presenti nel sistema. In particolare:

- Azione di scaling out intrapresa se il numero di server attivi è minore del valore $\left\lceil \frac{\#jobs}{SCALING_OUT_THR} \right\rceil$.
- Azione di scaling in intrapresa se il numero di server attivi è maggiore del valore $\left\lceil \frac{\#jobs}{SCALING_IN_THR} \right\rceil$.

2. Viene verificato che è possibile eseguire l'operazione di scaling:

- Per poter eseguire un'operazione di scale out devono essere presenti dei web server allocabili, cioè nello stato `TO_BE_REMOVED` o `REMOVED`.
- Per poter eseguire un'operazione di scale in devono essere presenti più di 2 web server allocati (nello stato `ACTIVE` o `TO_BE_ACTIVE`), poiché almeno un web server deve sempre essere acceso.

3. Si stabilisce il web server su cui applicare l'azione di scaling tramite i seguenti criteri:

- Se l'azione è di scale-out, si preferiscono i web server nello stato `TO_BE_REMOVED` per evitare i tempi di accensione.
- Se l'azione è di scale-in, si preferiscono i web server nello stato `TO_BE_ACTIVE` in modo da non dover aspettare che un server si scarichi.

12.2.3 Tempi di accensione

I tempi di accensione sono stati modellati utilizzando una variabile aleatoria Normale di media $5s$ e varianza $0.5s^2$. Il tempo di accensione è stato modellato allo stesso modo per tutti i server. Questo perché si assume che i web server siano omogenei e quindi ci si aspetta una bassa varianza per il tempo di accensione.

La variabile aleatoria Normale utilizzata è stata ottenuta mediante la **trasformazione di Box-Muller** che permette di ottenere una variabile aleatoria normale standard Z a partire da due distribuzioni uniformi, U_1 e U_2 , come segue:

$$\begin{aligned} Z &= \sqrt{-2 \log U_1} \cdot \cos(2 \pi U_2) \\ \text{oppure} \\ Z &= \sqrt{-2 \log U_1} \cdot \sin(2 \pi U_2) \end{aligned}$$

Per ottenere una variabile aleatoria normale con media e varianza generica è stata applicata la formula:

$$Z' = \mu + Z \sigma$$

Dove μ e σ sono la media e la deviazione standard che vogliamo applicare in modo che Z' sia una variabile aleatoria normale con parametri μ e σ^2 .

13 Modello computazionale migliorativo

13.1 La gestione degli eventi

L'introduzione dell'autoscaling richiede:

1. Una modifica negli eventi di arrivo e di completamento.
2. La gestione di nuovi eventi rispetto al modello base, ovvero:
 - Scaling in.
 - Scaling out.
 - Scaling out request.

13.1.1 Evento di arrivo

Rispetto al modello base, quando si verifica un evento di arrivo è necessario:

1. Aggiornare gli indicatori che vengono utilizzati per prendere decisioni di scaling.
2. Se è necessaria un'azione di scaling va schedata.

13.1.2 Evento di completamento

Rispetto al modello base, quando si verifica un evento di completamento è necessario:

1. Aggiornare gli indicatori che vengono utilizzati per prendere decisioni di scaling.
2. Se è necessaria un'azione di scaling va schedata.
3. Dopo aver calcolato l'avanzamento dei job, se un web server che si trova nello stato `TO_BE_REMOVED` non ha richieste pendenti viene portato nello stato `REMOVED`.

13.1.3 La funzione che pianifica lo scaling

Quando viene eseguita la simulazione del modello migliorativo la funzione `planScaling()` in figura 23, pianifica un evento di scaling in oppure fa una richiesta di scaling out.

13.1.4 Evento richiesta di scaling out

L'accensione di un server non è un'operazione immediata. Per questo motivo, quando è necessaria un'operazione di scaling out, viene fatta una richiesta. Il server scelto per servire l'operazione di scale out deve rispettare le seguenti proprietà:

- Come prima cosa si cerca un server nello stato `TO_BE_REMOVED`: ovvero un server per cui è stato richiesto lo spegnimento ma sta ancora finendo di servire i job. Questo permetterebbe di risparmiare il tempo di accensione.
- Se non ci sono server in fase di disattivazione, si sceglie quello con indice minimo che si trova nello stato `REMOVED`.

Tale server, se disponibile verrà portato nello stato TO_BE_ACTIVE e verrà settato come istante di attivazione un tempo pari alla somma tra:

- L'istante in cui è stata triggerata la richiesta.
- Un tempo campionato dalla distribuzione normale che modella il tempo di accensione di un server.

La richiesta di scaling out permette di gestire il fatto che il tempo di attivazione di un server non è deterministico. Quindi, se due richieste di scaling out sono molto vicine, più server possono trovarsi nello stato TO_BE_ACTIVE, ed è possibile che l'ordine delle richieste non corrisponda all'ordine delle effettive accensioni. Per cui, dopo aver calcolato il momento dell'accensione del server da allocare, viene ricalcolato il server che ha il tempo d'accensione più vicino in termini temporali e si schedula un evento di scaling out. Il codice che gestisce le richieste di scaling out è riportato in figura 24.

```
private void planScaling(SystemState s, double endTs,
double scalingIndicator){
    Infrastructure infrastructure = s.getInfrastructure();
    if (SCALING_OUT_THRESHOLD != INFINITY){
        boolean scalingOutPossible;
        boolean scalingOutCondition;
        boolean scalingInPossible;
        boolean scalingInCondition;
        int activatedServer =
            infrastructure.getNumWebServersByState(ServerState.ACTIVE) +
            infrastructure.getNumWebServersByState(ServerState.TO_BE_ACTIVE);

        scalingOutPossible = activatedServer < MAX_NUM_SERVERS;
        scalingInPossible =
            infrastructure.getNumWebServersByState(ServerState.ACTIVE) > 1;

        if (SCALING_INDICATOR_TYPE.equals("r0")) {
            scalingOutCondition =
                scalingIndicator > SCALING_OUT_THRESHOLD * 1.5;
            scalingInCondition =
                scalingIndicator < SCALING_OUT_THRESHOLD * 0.5;
        } else if (SCALING_INDICATOR_TYPE.equals("jobs")) {
            double scalingInThr = SCALING_OUT_THRESHOLD * 0.6;

            int requiredServerForScaleOut =
                (int) (Math.ceil(scalingIndicator / SCALING_OUT_THRESHOLD));
            int requiredServerForScaleIn =
                (int) (Math.ceil(scalingIndicator / scalingInThr));

            scalingOutCondition = activatedServer < requiredServerForScaleOut;
            scalingInCondition = activatedServer > requiredServerForScaleIn;
        } else {
            throw new IllegalArgumentException(...);
        }

        if (scalingOutCondition && scalingOutPossible)
            s.addEvent(new ScalingOutReqEvent(endTs));
        else if (scalingInCondition && scalingInPossible)
            s.addEvent(new ScalingInEvent(endTs));
    }
}
```

Figura 23. Metodo che implementa la pianificazione dello scaling

```

public void visit(SystemState s, ScalingOutReqEvent event)
throws IllegalLifeException {

    Infrastructure infrastructure = s.getInfrastructure();
    double endTs = event.getTimestamp();

    /* Generate time for turn on a web sever */
    double turnOnTime = s.getTurnOnVA().gen();

    // From request to effective scale out
    var serverTarget = infrastructure.requestScaleOut(endTs, turnOnTime);

    /* Find the earliest WS to make active: it needs to be done
       in case no scaling out event is already scheduled */
    WebServer nextScaleOut = infrastructure.findNextScaleOut();
    s.addEvent(
        new ScalingOutEvent(nextScaleOut.getActivationTimestamp(),
            nextScaleOut));

    // Schedule to INFINITY the next request
    s.addEvent(new ScalingOutReqEvent(INFINITY));

    /* Update the current system clock */
    s.setCurrent(endTs);
}

```

Figura 24. Metodo per le gestione delle richieste di scaling out

13.1.5 Evento di scaling out

Quando si verifica un evento di scaling out è necessario:

1. Recuperare il clock corrente.
2. Processare l'avanzamento dei job.
3. Portare nello stato ACTIVE il server per cui era stato richiesto di scalare.
4. Cercare il prossimo server per cui generare un evento di scaling out, preferendo prima i server nello stato TO_BE_REMOVED.

Il codice per la gestione degli eventi di scaling out è riportato in figura 25.

13.1.6 Evento di scaling in

Quando si verifica un evento di scaling in si cerca un server tale che:

- Si trova nello stato TO_BE_ACTIVE: ovvero che si sta attivando ma ancora non è effettivamente attivo.

- Se nessun server è ancora in fase di accensione, se ne cerca uno nello stato **ACTIVE**.

Il server individuato dovrà essere portato nello stato **TO_BE_REMOVED** se ancora possiede dei job da eseguire, altrimenti viene portato direttamente nello stato **REMOVED**. Il codice per la gestione degli eventi di scaling in è riportato in figura 26.

13.2 La variabile aleatoria Normale

La variabile aleatoria normale è stata implementata utilizzando la **trasformazione di Box-Muller** che permette di ottenere una distribuzione normale standard a partire da due distribuzioni uniformi, come descritto nel modello delle specifiche (12.2.3). Dunque, per implementare la variabile aleatoria normale sono state utilizzate due variabili aleatorie uniformi che utilizzano gli stream 6 e 7.

```
public void visit(SystemState s, ScalingOutEvent event)
throws IllegalLifeException {

    Infrastructure infrastructure = s.getInfrastructure();

    /* Get the current clock and the one of this arrival */
    double startTs = s.getCurrent();
    double endTs = event.getTimestamp();

    /* Advance job execution */
    infrastructure.computeJobsAdvancement(startTs, endTs, false);

    /* Set server to be effectively active */
    infrastructure.scaleOut(endTs, event.getTarget());

    /* Find the earliest WS to make active */
    WebServer nextScaleOut = infrastructure.findNextScaleOut();
    double nextActivationTS = nextScaleOut == null ?
        INFINITY :
        nextScaleOut.getActivationTimestamp();

    s.addEvent(new ScalingOutEvent(nextActivationTS, nextScaleOut));

    /* Generate next completion */
    double nextCompletionTs = infrastructure.activeJobExists() ?
        infrastructure.computeNextCompletionTs(endTs) : INFINITY;

    Event nextCompletion = new CompletionEvent(nextCompletionTs);
    s.addEvent(nextCompletion);

    /* Update the current system clock */
    s.setCurrent(endTs);
}
```

Figura 25. Metodo per la gestione di eventi di scaling out

```

public void visit(SystemState s, ScalingInEvent event)
throws IllegalLifeException {

    Infrastructure infrastructure = s.getInfrastructure();
    double endTs = event.getTimestamp();

    infrastructure.scaleIn(endTs);

    s.addEvent(new ScalingInEvent(INFINITY));

    /* Generate next completion */
    double nextCompletionTs = infrastructure.activeJobExists() ?
        infrastructure.computeNextCompletionTs(endTs) :
        INFINITY;
    Event nextCompletion = new CompletionEvent(nextCompletionTs);
    s.addEvent(nextCompletion);

    s.setCurrent(endTs);
}

```

Figura 26. Metodo per la gestione di eventi di scaling in

14 Validazione del modello migliorativo

Per il modello migliorativo si vuole validare l'autoscaler e capire se effettivamente alloca i server in corrispondenza dei picchi di carico.

14.1 Studio 1

La configurazione utilizzata per questo esperimento è la seguente:

- Seed pari a 42.
- Lo spike server è acceso con $SI^{\max} = 3$.
- Il numero massimo di web server è pari a 3.
- Tutti i server hanno una capacità di $1 \frac{\text{op.}}{\text{s}}$.
- Tempi di interarrivo iperesponenziali di media 0.15 e coefficiente di variazione pari a 4.
- Domande dei job iperesponenziali di media 0.16 e coefficiente di variazione pari a 4.
- Fluttuazioni long term presenti.
- Indice per prendere decisioni di scaling basato sul tempo di risposta globale dei web server.

Un estratto del comportamento dell'autoscaler basato sul tempo di risposta è stato riportato in figura 27. Questo grafico mostra che il numero dei server cresce in corrispondenza dei picchi di carico.

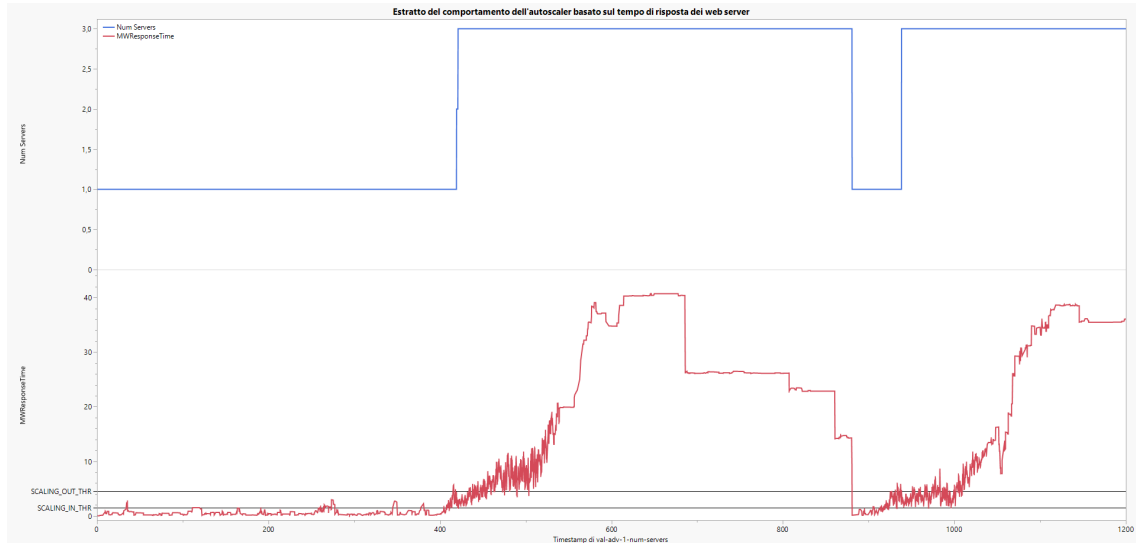


Figura 27. Estratto del comportamento dell'autoscaler basato sul tempo di risposta

14.2 Studio 2

Il secondo studio è stato effettuato utilizzando la stessa configurazione del precedente. L'unica differenza è che l'indicatore per prendere le decisioni di scaling è basato sul numero di job nel sistema. Un estratto del comportamento di questo autoscaler è riportato in figura 28. Anche in questo caso i server vengono allocati in corrispondenza dei picchi di carico.

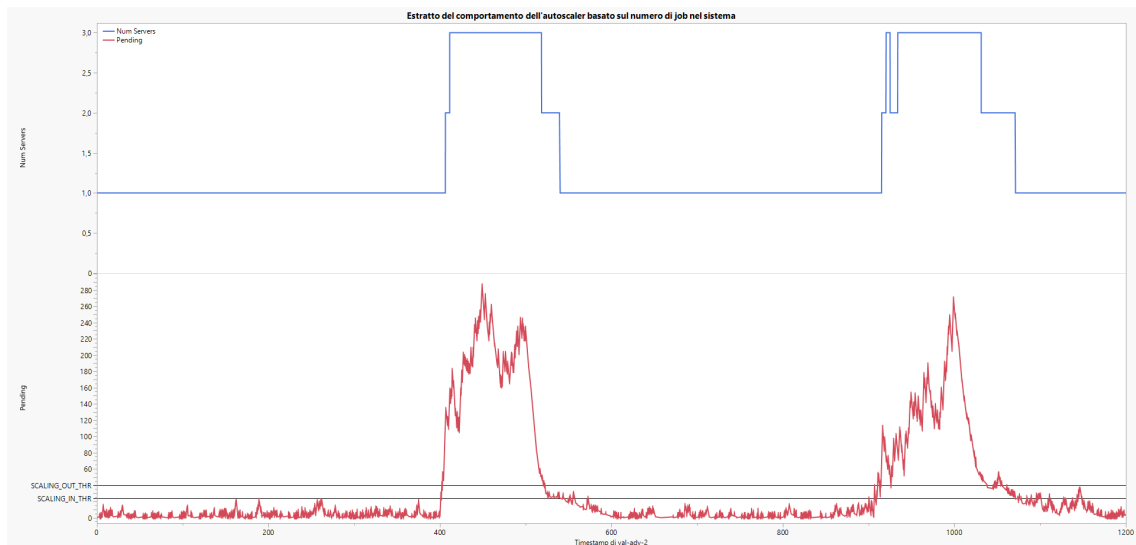


Figura 28. Estratto del comportamento dell'autoscaler basato sul numero di job nel sistema

15 Studio del transitorio

In questa sezione viene riportato lo studio del transitorio per alcune configurazioni. Questo ci permette di assicurare che i parametri utilizzati negli esperimenti successivi corrispondo a delle simulazioni le cui statistiche convergono. I parametri fissati in tutti gli studi sono riportati nella tabella 5. Inoltre, per lo studio del transitorio del modello avanzato sono state attivate anche le fluttuazioni long term. Quest'ultime vengono generate in periodi di $500s$ in cui:

- Per un periodo di $400s$, il tasso degli arrivi ha una media $\lambda = \frac{1}{0.15}$.
- Per un periodo di $100s$, il tasso degli arrivi ha una media $\lambda_{\text{fast}} = \frac{4}{0.15}$.

15.1 Studio 1

In questo primo studio viene utilizzata la seguente configurazione:

- Il numero massimo di web server è pari a 3.
- Indice per prendere decisioni di scaling basato sul tempo di risposta globale dei web server.
- Soglia per prendere decisioni di scaling out pari a $4.5s$
- Soglia per prendere decisioni di scaling in pari a $1.5s$
- Lo spike server è acceso con $SI^{\text{max}}=3$.

I risultati in forma grafica sono riportati in figura 29. Invece, i risultati in forma tabellare, ottenuti con 32 repliche e intervalli di confidenza al 95% utilizzando la distribuzione di Student sono riportati in tabella 16.

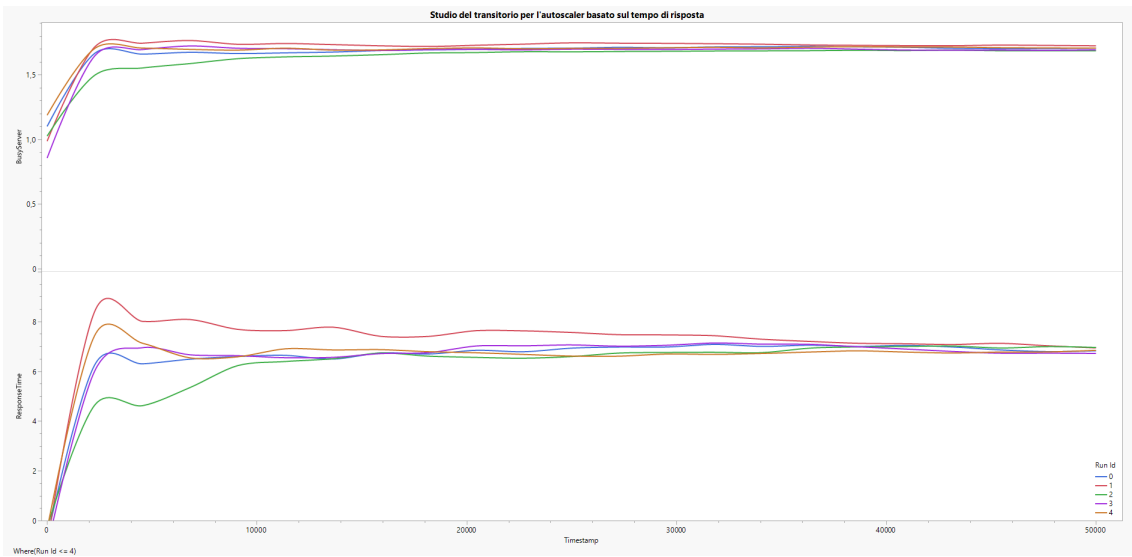


Figura 29. Studio del transitorio per l'autoscaler basato sul tempo di risposta

Tempo di risposta	Utilizzazione
$6,573755 \pm 0,073996$	$1,700062 \pm 0,004720$

Tabella 16. Intervalli di confidenza al 95% calcolati con 32 ripetizioni e la distribuzione di Student

	Stream			
Id della config.	0	1	2	3
0	42	962850	598499780	334435817
1	1824521676	1995872371	1610839585	2068794600
2	858081894	1360287319	2144255119	1704311751
3	354527426	1125755613	475541493	525930673
4	1405335934	1632204161	2139047438	936635421

Tabella 17. Prima parte dei seed iniziali di ciascuna run

	Stream			
Id della config.	4	5	6	7
0	424484935	1068730318	1611527	437034476
1	1214379786	1939207237	1500279549	1958054120
2	289482278	104067155	71579530	283218942
3	1502637497	1371593703	709760504	1923444528
4	837633888	653348813	514554060	24152529

Tabella 18. Seconda parte dei seed iniziali di ciascuna run

15.2 Studio 2

Nel secondo studio viene utilizzata la seguente configurazione:

- Il numero massimo di web server è pari a 3.
- Indice per prendere decisioni di scaling basato sul numero di job nel sistema.
- Soglia per prendere decisioni di scaling out pari a 40 job.
- Soglia per prendere decisioni di scaling in pari a 24 job.
- Lo spike server è acceso con $SI^{\max}=3$.

I risultati in forma grafica sono riportati in figura 30. Invece, i risultati in forma tabellare, ottenuti con 32 repliche e intervalli di confidenza al 95% utilizzando la distribuzione di Student sono riportati in tabella 19.

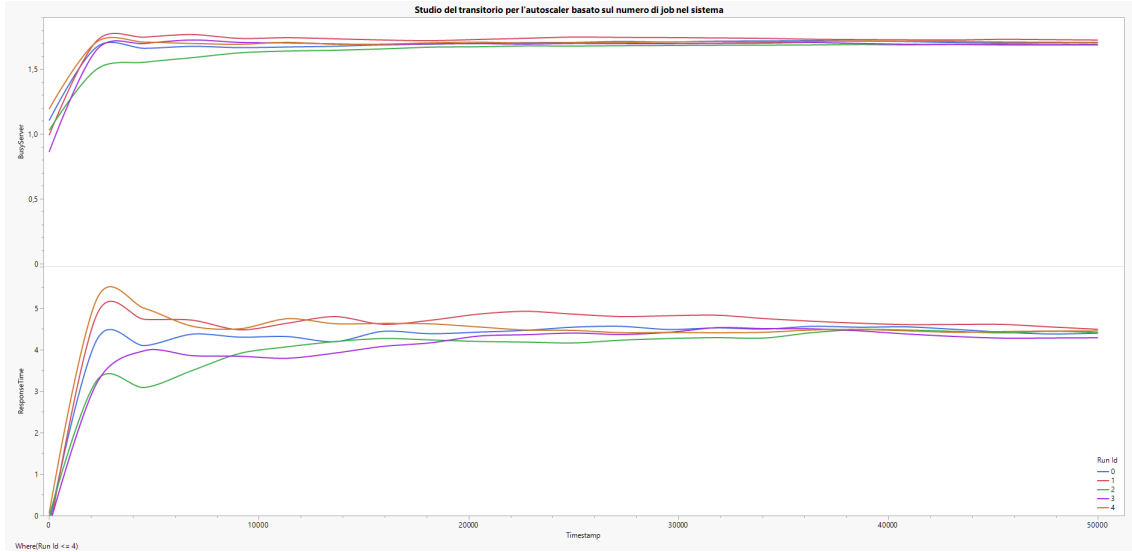


Figura 30. Studio del transitorio per l'autoscaler basato sul numero di job nel sistema

Tempo di risposta	Utilizzazione
$4,227739 \pm 0,055999$	$1,700521 \pm 0,004735$

Tabella 19. Intervalli di confidenza al 95% calcolati con 32 ripetizioni e la distribuzione di Student

Id della config.	Stream			
	0	1	2	3
0	42	962850	598499780	334435817
1	1824521676	1995872371	1610839585	2068794600
2	858081894	1360287319	2144255119	1704311751
3	354527426	1125755613	475541493	525930673
4	1405335934	1632204161	2139047438	936635421

Tabella 20. Prima parte dei seed iniziali di ciascuna run

Id della config.	Stream			
	4	5	6	7
0	424484935	1068730318	1611527	437034476
1	1214379786	1939207237	1965992840	1146557411
2	289482278	104067155	1060484723	2097390735
3	1502637497	1371593703	1969391369	1748423444
4	837633888	653348813	2077773969	1780948865

Tabella 21. Seconda parte dei seed iniziali di ciascuna run

16 Esperimenti sul modello migliorativo

In questa sezione vengono descritti gli esperimenti ad orizzonte finito e quelli ad orizzonte infinito. Gli studi ad orizzonte infinito vengono utilizzati per valutare i parametri dell'autoscaler. La scelta dell'orizzonte infinito, piuttosto che quello finito, è dovuta al fatto che si vogliono identificare dei parametri che vadano bene per tutta la durata della simulazione e non in uno specifico intervallo di tempo. Gli studi ad orizzonte finito invece sono stati condotti per mostrare il comportamento dei due autoscaler durante un periodo di fluttuazione e nei successivi istanti, utilizzando le soglie identificate tramite gli studi ad orizzonte finito.

16.1 Studio 1

Nel primo studio ad orizzonte infinito si vogliono valutare diverse configurazioni dell'autoscaler basato sul tempo di risposta. La configurazione utilizzata è la seguente:

- Seed pari a 42.
- Lo spike server è acceso e vengono provate le seguenti soglie SI^{max} : 10, 50, 90, 130, 160.
- Il numero massimo di web server è pari a 3.
- Tutti i server hanno una capacità di $1 \frac{op}{s}$.
- Tempi di interarrivo iperesponenziali di media 0.15 e coefficiente di variazione pari a 4.
- Domande dei job iperesponenziali di media 0.16 e coefficiente di variazione pari a 4.
- Fluttuazioni long term presenti.
- Indice per prendere decisioni di scaling basato sul tempo di risposta globale dei web server.
- Batch mean con 16 batch di taglia 32768.

I risultati, riportati in figura 31 ed in tabella 22, mostrano che nell'ottica di minimizzare il tempo di risposta la configurazione con $SCALING_OUT_THR = 1.5$, $SCALING_IN_THR = 0.5$ ed $SI_MAX = 10$ risulta essere la migliore. Questo risultato è in linea con le aspettative in quanto, l'autoscaler configurato in questo modo tende ad aggiungere server più facilmente e ad utilizzare di più lo spike server. Tuttavia, lo spike server, rappresenta il secondo livello dell'autoscaler gerarchico, ed è pensato solamente per gestire brevi periodo di carico. In questa ottica, si potrebbe preferire la configurazione con $SCALING_OUT_THR = 1.5$, $SCALING_IN_THR = 0.5$ ed $SI_MAX = 50$ che porta lo spike server ad un'utilizzazione in linea con quella ottenuta con la configurazione scelta per il modello base. Quest'ultima porta ad un tempo di risposta di circa 8s e ad un'utilizzazione dello spike server circa di 0.06. La configurazione con $SCALING_OUT_THR = 1.5$, $SCALING_IN_THR = 0.5$ ed $SI_MAX = 50$, oltre a ridurre l'utilizzazione dello spike server, porta ad un tempo di risposta circa 4 volte inferiore rispetto al modello base.

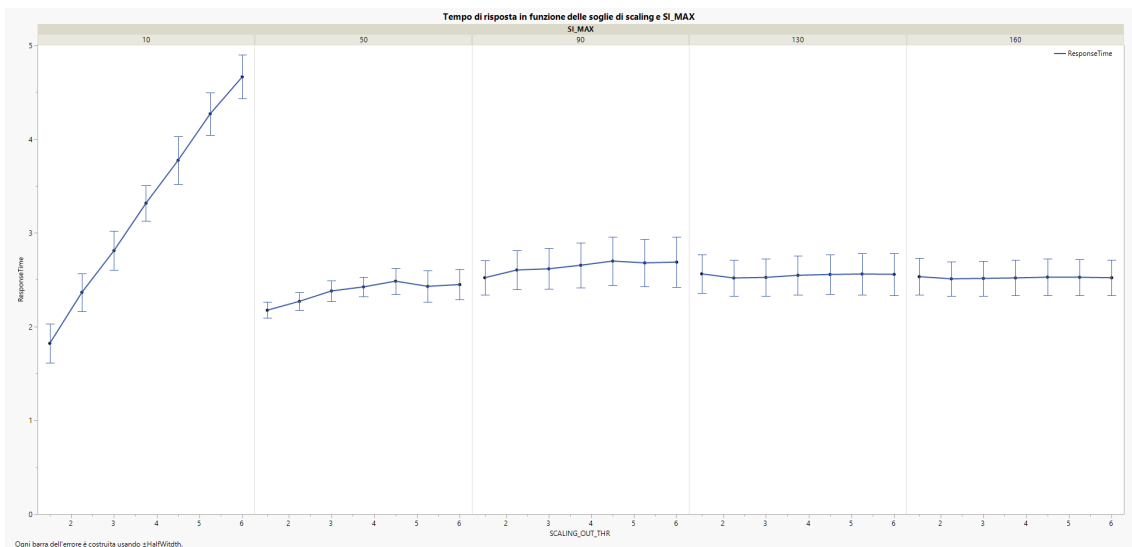


Figura 31. Tempo di risposta in funzione della soglia di scaling e SI^{max} per l'autoscaler basato sul tempo di risposta dei web server

Scaling out thr.	Scaling in thr.	SI_MAX	Tempo di risposta	Utilizzazione SS
1.5	0.5	10	1,822350 $\pm$ 0,210817	0,280135 $\pm$ 0,019064
1.5	0.5	50	2,175501 $\pm$ 0,084896	0,053676 $\pm$ 0,009020
1.5	0.5	90	2,522265 $\pm$ 0,183272	0,008154 $\pm$ 0,004493
1.5	0.5	130	2,563056 $\pm$ 0,207758	0,000993 $\pm$ 0,001406
1.5	0.5	160	2,533172 $\pm$ 0,197079	0,000064 $\pm$ 0,000177
2.25	0.75	10	2,365661 $\pm$ 0,200329	0,311502 $\pm$ 0,016982
2.25	0.75	50	2,270064 $\pm$ 0,096851	0,066850 $\pm$ 0,010372
2.25	0.75	90	2,604171 $\pm$ 0,209386	0,010528 $\pm$ 0,003850
2.25	0.75	130	2,519198 $\pm$ 0,191542	0,001207 $\pm$ 0,001752
2.25	0.75	160	2,509883 $\pm$ 0,184967	0,000189 $\pm$ 0,000525
3	1	10	2,810282 $\pm$ 0,208329	0,331912 $\pm$ 0,012214
3	1	50	2,380744 $\pm$ 0,112020	0,077775 $\pm$ 0,012763
3	1	90	2,617349 $\pm$ 0,217594	0,013377 $\pm$ 0,003576
3	1	130	2,525384 $\pm$ 0,198054	0,001432 $\pm$ 0,001917
3	1	160	2,514731 $\pm$ 0,186467	0,000190 $\pm$ 0,000527
3.75	1.25	10	3,316838 $\pm$ 0,189995	0,347809 $\pm$ 0,012348
3.75	1.25	50	2,424116 $\pm$ 0,102699	0,096741 $\pm$ 0,007172
3.75	1.25	90	2,654998 $\pm$ 0,239044	0,017175 $\pm$ 0,004095
3.75	1.25	130	2,548002 $\pm$ 0,209379	0,001895 $\pm$ 0,002759
3.75	1.25	160	2,520038 $\pm$ 0,188745	0,000254 $\pm$ 0,000705
4.5	1.5	10	3,773531 $\pm$ 0,255449	0,360549 $\pm$ 0,012919
4.5	1.5	50	2,485224 $\pm$ 0,139620	0,104193 $\pm$ 0,004328
4.5	1.5	90	2,699398 $\pm$ 0,258137	0,020695 $\pm$ 0,006842
4.5	1.5	130	2,556480 $\pm$ 0,214120	0,002255 $\pm$ 0,003041
4.5	1.5	160	2,528283 $\pm$ 0,193546	0,000173 $\pm$ 0,000481
5.25	1.75	10	4,269174 $\pm$ 0,229252	0,375147 $\pm$ 0,014511
5.25	1.75	50	2,429914 $\pm$ 0,169450	0,122912 $\pm$ 0,008966
5.25	1.75	90	2,680125 $\pm$ 0,250249	0,021751 $\pm$ 0,005142
5.25	1.75	130	2,561771 $\pm$ 0,221388	0,002470 $\pm$ 0,003606
5.25	1.75	160	2,526980 $\pm$ 0,192689	0,000135 $\pm$ 0,000376
6	2	10	4,663234 $\pm$ 0,233434	0,385512 $\pm$ 0,014230
6	2	50	2,448899 $\pm$ 0,160175	0,133653 $\pm$ 0,014650
6	2	90	2,688452 $\pm$ 0,267180	0,026293 $\pm$ 0,007409
6	2	130	2,558032 $\pm$ 0,222495	0,002905 $\pm$ 0,003606
6	2	160	2,522022 $\pm$ 0,189763	0,000475 $\pm$ 0,001320

Tabella 22. Tempo di risposta in funzione della soglia di scaling e $SI^{\max}$ per l'autoscaler basato sul tempo di risposta dei web server.

16.2 Studio 2

Nel secondo studio ad orizzonte infinito si vogliono valutare diverse configurazioni dell'autoscaler basato sul numero di job nel sistema. La configurazione utilizzata è la seguente:

- Seed pari a 42.
- Lo spike server è acceso e vengono provate le seguenti soglie $SI^{\max}$: 10, 50, 90, 130, 160.
- Il numero massimo di web server è pari a 3.
- Tutti i server hanno una capacità di $1 \frac{op}{s}$.
- Tempi di interarrivo iperesponenziali di media 0.15 e coefficiente di variazione pari a 4.
- Domande dei job iperesponenziali di media 0.16 e coefficiente di variazione pari a 4.
- Fluttuazioni long term presenti.
- Indice per prendere decisioni di scaling basato sul numero di job nel sistema.
- Batch mean con 16 batch di taglia 32768.

I risultati per l'autoscaler basato sul numero di job nel sistema sono riportati nella figura 32 e nella tabella 23. In questo caso, il tempo di risposta minore si ottiene con la configurazione con $SCALING_OUT_THR = 5$, $SCALING_IN_THR = 3$ ed $SI_MAX = 10$. Tuttavia, per ottenere un tempo così basso lo spike server viene utilizzato di più rispetto a quello per cui è stato introdotto. Tutte le altre configurazioni con $SI^{\max} = 10$ tendono ad aggiungere server più raramente, utilizzando ancora di più lo spike server. In questo caso, la soluzione che fornisce il tempo di risposta minore utilizzando il giusto lo spike server è quella con $SCALING_OUT_THR = 5$, $SCALING_IN_THR = 3$ ed $SI_MAX = 50$.

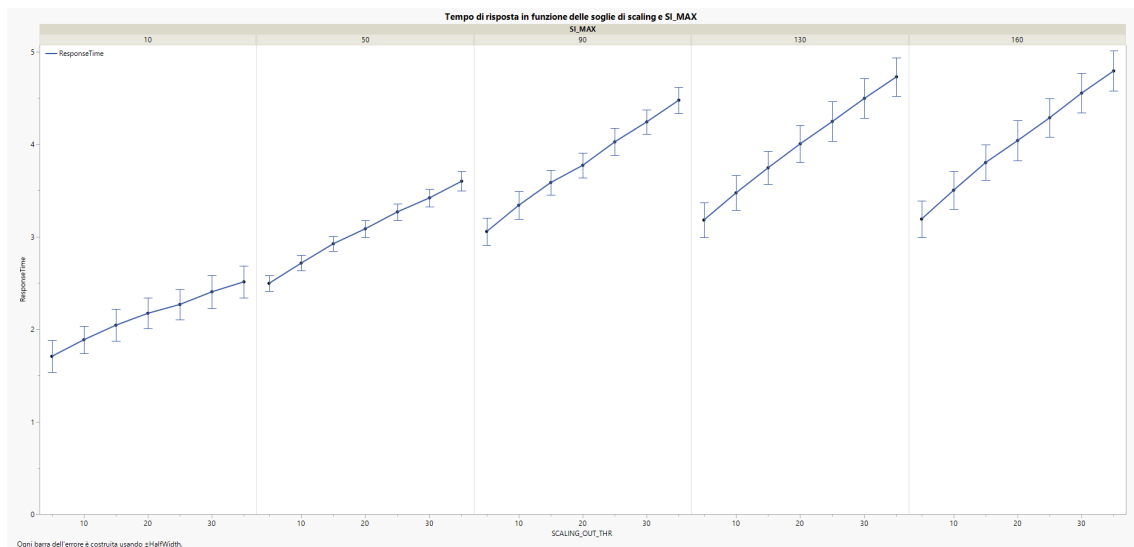


Figura 32. Tempo di risposta in funzione delle soglie di scaling e $SI^{\max}$ per l'autoscaler basato sul numero di job nel sistema

Scaling out thr.	Scaling in thr.	SI_MAX	Tempo di risposta	Utilizzazione SS
5	3	10	1,706649 ± 0,170306	0,375057 ± 0,018067
5	3	50	2,495689 ± 0,088648	0,059968 ± 0,009816
5	3	90	3,056163 ± 0,144805	0,009128 ± 0,003503
5	3	130	3,180195 ± 0,186169	0,001082 ± 0,000841
5	3	160	3,190866 ± 0,198639	0,000185 ± 0,000446
10	6	10	1,886099 ± 0,146931	0,440728 ± 0,018427
10	6	50	2,714132 ± 0,082964	0,067805 ± 0,010341
10	6	90	3,339396 ± 0,149440	0,011106 ± 0,002762
10	6	130	3,473642 ± 0,188139	0,002046 ± 0,000690
10	6	160	3,502210 ± 0,201755	0,000321 ± 0,000352
15	9	10	2,043275 ± 0,171548	0,477564 ± 0,014699
15	9	50	2,923487 ± 0,078250	0,077289 ± 0,009936
15	9	90	3,585479 ± 0,132565	0,013652 ± 0,004078
15	9	130	3,744868 ± 0,178206	0,003370 ± 0,001516
15	9	160	3,801522 ± 0,192783	0,000677 ± 0,001119
20	12	10	2,172038 ± 0,165814	0,490787 ± 0,016526
20	12	50	3,085824 ± 0,092797	0,085928 ± 0,008662
20	12	90	3,771782 ± 0,134750	0,017366 ± 0,004840
20	12	130	4,003923 ± 0,199025	0,003640 ± 0,001847
20	12	160	4,039351 ± 0,216069	0,001137 ± 0,001103
25	15	10	2,267487 ± 0,163662	0,505298 ± 0,017796
25	15	50	3,268429 ± 0,090224	0,095224 ± 0,007677
25	15	90	4,025079 ± 0,148128	0,020156 ± 0,005712
25	15	130	4,245488 ± 0,212790	0,003489 ± 0,001957
25	15	160	4,285613 ± 0,205433	0,001165 ± 0,001492
30	18	10	2,404179 ± 0,179015	0,512597 ± 0,015019
30	18	50	3,419652 ± 0,095060	0,108178 ± 0,009134
30	18	90	4,241159 ± 0,133381	0,022164 ± 0,005692
30	18	130	4,494907 ± 0,213485	0,005147 ± 0,003013
30	18	160	4,552775 ± 0,214975	0,001587 ± 0,001260
35	21	10	2,512271 ± 0,171225	0,520974 ± 0,017405
35	21	50	3,599571 ± 0,105053	0,126702 ± 0,008550
35	21	90	4,475589 ± 0,139795	0,025195 ± 0,005132
35	21	130	4,728163 ± 0,207006	0,005919 ± 0,002158
35	21	160	4,791817 ± 0,216247	0,001654 ± 0,000639

Tabella 23. Tempo di risposta in funzione della soglia di scaling e $SI^{\max}$ per l'autoscaler basato sul numero di job nel sistema

16.3 Studio 3

L'ultimo studio è ad orizzonte finito e vuole mostrare il comportamento dei due autoscaler utilizzando le soglie individuate nei precedenti studi ed effettuando un confronto con il caso in cui il meccanismo di autoscaling orizzontale non è presente. La configurazione utilizzata è la seguente:

- Seed pari a 42.
- Il numero massimo di web server è pari a 3.
- Tutti i server hanno una capacità di $1 \frac{op}{s}$.
- Tempi di interarrivo iperesponenziali di media 0.15 e coefficiente di variazione pari a 4.
- Domande dei job iperesponenziali di media 0.16 e coefficiente di variazione pari a 4.

- Fluttuazioni long term presenti.
- 32 ripetizioni da 900 secondi: prolungando la simulazione (di 400s) prima che si verifichi la successiva fluttuazione long-term è possibile osservare anche come vengono smaltiti i job accumulati durante il periodo di fluttuazione.

In uno studio ad orizzonte finito sono importanti le condizioni iniziali che in questo caso sono:

- Sistema vuoto: per dare una maggiore enfasi al picco del tasso di arrivo. Infatti se si fosse partiti con dei job in coda le performance sarebbero state influenzate anche dallo smaltimento della coda.
- Inizialmente c'è solo un web server acceso insieme allo spike server.

L'andamento del tempo di risposta e degli arrivi sui 900 secondi è riportato in figura 33, da cui si può osservare che i due meccanismi di autoscaling riescono a controllare perfettamente il tempo di risposta durante il periodo di congestione. Il grafico mostra delle medie (calcolate tramite delle finestre temporali) del tempo di risposta calcolato su 32 run. Inoltre, utilizzando la distribuzione di Student sono stati calcolati gli intervalli di confidenza riportati in tabella 24. Le configurazioni dei due autoscaler risultano comparabili anche in termini di utilizzazione dello spike server.

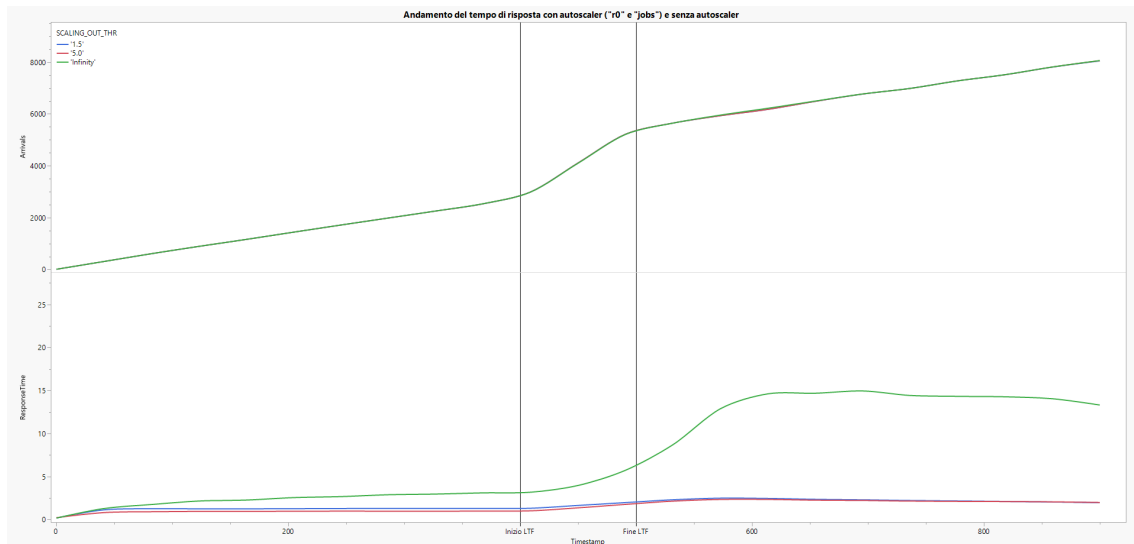


Figura 33. Andamento del tempo di risposta con autoscaler ("r0" e "jobs") e senza autoscaler

T	Scaling out thr.	Scaling in thr.	SI_MAX	Tempo di risposta	Utilizzazione SS
j	5	3	50	$1,993114 \pm 0,101082$	$0,041779 \pm 0,010606$
r0	1.5	0.5	50	$1,990622 \pm 0,095114$	$0,050854 \pm 0,009231$
n	INFINITY	INFINITY	50	$13,487828 \pm 1,445137$	$0,424181 \pm 0,031847$

Tabella 24. Intervalli di confidenza per il tempo di risposta ottenuti con 32 ripetizioni e distribuzione di Student. La colonna T indica il tipo di autoscaler (j=jobs e n=nessuno).

17 Conclusioni

17.1 La scelta della soglia $SI^{\max}$ nel modello base

Nel modello base si è visto come aumentare la soglia $SI^{\max}$ peggiori le performance del sistema. Questo perché all'aumentare di questo valore lo spike server viene utilizzato sempre meno. Tuttavia, lo spike server deve essere utilizzato solamente per gestire le congestioni dei web server. Per questo motivo nello primo studio del modello base si richiede un tempo target pari ad $8s$ ottenuto in corrispondenza di $SI^{\max} = 90$.

17.2 L'impatto del tasso di arrivo sullo spike server

Sono state individuate le seguenti classi di carico:

- Basso: $1 \frac{\text{req.}}{s} \leq \lambda \leq 6 \frac{\text{req.}}{s}$:
 - Il tempo di risposta del sistema è sempre inferiore ai $10s$ per qualsiasi valore di $SI^{\max}$.
 - In assenza dello spike server il passaggio da $6 \frac{\text{req.}}{s}$ a $7 \frac{\text{req.}}{s}$ causa un aumento importante del tempo di risposta.
- Medio: $6 \frac{\text{req.}}{s} < \lambda \leq 10 \frac{\text{req.}}{s}$:
 - Fino al tasso di arrivo di $8 \frac{\text{req.}}{s}$: aumento del tempo di risposta del sistema dovuto alla saturazione del web server.
 - Da $8 \frac{\text{req.}}{s}$ a $10 \frac{\text{req.}}{s}$: aumenta il numero di violazioni rivelate dal load controller che manda più richieste verso lo spike server facendo decrescere il tempo di risposta del sistema.
- Alto: $10 \frac{\text{req.}}{s} < \lambda \leq 12 \frac{\text{req.}}{s}$: il tempo di risposta del sistema torna a crescere a causa della saturazione dello spike server.

17.3 Configurazioni degli autoscaler nel modello migliorativo

Per i due autoscaler, le configurazioni che hanno portato al minor tempo di risposta del sistema possibile sono riportate in figura 25. Tuttavia, si può osservare come questi parametri causino un sovrautilizzo dello spike server, che rappresenta il secondo livello dell'autoscaler gerarchico. Un ottimo compromesso tra il tempo di risposta e l'utilizzazione dello spike server si ottiene con le configurazioni riportate in tabella 26.

T	Scaling out thr.	Scaling in thr.	SI_MAX	Tempo di risposta	Utilizzazione SS
j	5	3	10	$1,706649 \pm 0,170306$	$0,375057 \pm 0,018067$
r0	1.5	0.5	10	$1,822350 \pm 0,210817$	$0,280135 \pm 0,019064$

Tabella 25. Configurazioni dell'autoscaler corrispondenti ai tempi di risposta minori. La colonna T indica il tipo di autoscaler (j=jobs e n=nessuno).

T	Scaling out thr.	Scaling in thr.	SI_MAX	Tempo di risposta	Utilizzazione SS
j	1.5	0.5	50	$2,175501 \pm 0,084896$	$0,053676 \pm 0,009020$
r0	5	3	50	$2,495689 \pm 0,088648$	$0,059968 \pm 0,009816$

Tabella 26. Configurazioni dell'autoscaler che offrono un ottimo compromesso tra utilizzazione dello spike server e tempo di risposta. La colonna T indica il tipo di autoscaler (j=jobs e n=nessuno).

17.4 Comportamento delle soglie durante il periodo di fluttuazione

Con lo studio ad orizzonte finito su $900s$ (di cui $400s$ con carico medio, $100s$ con carico alto e poi altri $400s$ con carico medio) si può osservare come i due meccanismi di scaling con le soglie impostate riescano a controllare il tempo di risposta rispetto al caso senza autoscaler. Inoltre, si può osservare che con le soglie scelte non c'è un autoscaler che si comporta nettamente meglio dell'altro.

18 Appendice

18.1 Validazione manuale dello stato del sistema

Esempio 4. Si pensi di avere un server che adotta uno scheduling di tipo processor sharing. Il server ha una capacità $C = 0.5 \frac{\text{job}}{s}$. All'istante $t = 0$ il centro è vuoto. Si assuma che il primo arrivo si verifichi all'istante $t = 2s$. Il job 1 che arriva a questo istante richiede $r_1(0) = 5s$. Fino all'istante $t = 4s$, in cui si verifica l'arrivo successivo, il job usufruisce dell'intera capacità del server. In questo periodo lungo $\Delta t = 2$ il tempo di servizio rimanente viene decrementato come segue:

$$\begin{aligned} r_1(4) &= r_1(0) - 0.5 \cdot 2 \\ &= 5 - 1 = 4s \end{aligned}$$

Nel momento in cui arriva il secondo job, che richiede $r_2(4) = 7s$, la capacità per ciascun job viene dimezzata a $v(4) = 0.25 \frac{\text{job}}{s}$. Il primo job a terminare sarà quello con un tempo di servizio rimanente minore. Dunque, il primo completamento sarà quello del job 1. Si può calcolare l'istante t^* in cui questo completamento avviene tramite l'equazione:

$$\begin{aligned} r_1(t^*) &= r_1(4) - 0.25 (t^* - 4) = 0 \\ &\Downarrow \\ 0.25 (t^* - 4) &= r_1(4) \\ &\Downarrow \\ t^* &= 4r_1(4) + 4 = 20 \end{aligned}$$

Nell'istante t^* , il tempo di servizio rimanente del job 2 sarà:

$$\begin{aligned} r_2(20) &= r_2(4) - 0.25 (20 - 4) \\ &= 7 - 4 = 3s \end{aligned}$$

Il job 2 è rimasto da solo e quindi tornerà ad usufruire dell'intera capacità del server $v(20) = 0.5 \frac{\text{job}}{s}$. Il completamento di questo job avverrà all'istante $t^* = 26s$ come si può vedere con i seguenti calcoli:

$$\begin{aligned} r_2(t^*) &= r_2(20) - 0.5 (t^* - 20) \\ &\Downarrow \\ 0.5 (t^* - 20) &= r_2(20) \\ t^* &= 2r_2(20) + 20 = 26s \end{aligned}$$

Allo stesso risultato si poteva arrivare osservando che il server esegue mezzo job al secondo. Quindi per completare un job di $3s$ ci vogliono $6s$.

18.2 I test delle variabili aleatorie

Per testare che le variabili aleatorie implementate seguono le distribuzioni teoriche, sono stati costruiti degli intervalli di confidenza per la media campionaria utilizzando la disuguaglianza di Chebyshev:

$$P(|X - \mu| > \varepsilon) \geq \frac{\sigma_X^2}{\varepsilon^2}$$

Nel nostro caso X corrisponde alla media campionaria X_n per la quale sappiamo che:

$$\begin{aligned} E[X_n] &= \mu_X \\ D^2[X_n] &= \frac{\sigma_X^2}{n} \end{aligned}$$

Dove X è la variabile aleatoria che segue una certa distribuzione, dalla quale effettuiamo gli n campionamenti per calcolare poi la media campionaria. Dunque possiamo scrivere la disuguaglianza come segue:

$$\begin{aligned} P(|X_n - \mu_X| > \varepsilon) &\leq \frac{\sigma_X^2}{n \varepsilon^2} \\ P(|X_n - \mu_X| \leq \varepsilon) &\geq 1 - \frac{\sigma_X^2}{n \varepsilon^2} \\ P(-\varepsilon \leq X_n - \mu_X \leq \varepsilon) &\geq 1 - \frac{\sigma_X^2}{n \varepsilon^2} \\ P(X_n - \varepsilon \leq \mu_X \leq X_n + \varepsilon) &\geq 1 - \frac{\sigma_X^2}{n \varepsilon^2} \end{aligned}$$

Dunque abbiamo costruito un intervallo di confidenza per la media di una variabile aleatoria generica. Ovvero con una probabilità $\geq 1 - \frac{\sigma_X^2}{n \varepsilon^2}$ possiamo dire che il vero valore della media cade nell'intervallo $[X_n - \varepsilon, X_n + \varepsilon]$. Per fare il test vengono fissati il numero di ripetizioni n e la confidenza α . Dalla definizione di intervallo di confidenza fissiamo:

$$1 - \alpha = 1 - \frac{\sigma_X^2}{n \varepsilon^2} \Rightarrow \varepsilon = \sqrt{\frac{\mu_X^2 C^2}{n \alpha}}$$

In questo modo per ciascun n e α passati come parametri al test viene calcolato ε che ci permette di dire con probabilità $1 - \alpha$ che il vero valore della media della variabile aleatoria ricade nell'intervallo di confidenza.

Per quanto riguarda il vero valore della varianza dell'iperesponenziale si è utilizzata sempre la disuguaglianza di Chebyshev osservando che:

$$\mu_X = \sqrt{\frac{\sigma_X^2}{C^2}}$$

Dunque possiamo riscrivere la disuguaglianza come:

$$P(C^2 (X_n - \varepsilon^2) \leq \sigma_X^2 \leq C^2 (X_n + \varepsilon^2)) \geq 1 - \frac{\sigma_X^2}{n \varepsilon^2}$$

Analogamente al caso della media si calcola ε in funzione di α e n . In questo modo abbiamo costruito un intervallo di confidenza per la varianza dell'iperesponenziale che ci permette di dire che con probabilità $1 - \alpha$ il vero valore della varianza cade all'interno dell'intervallo.

Per la varianza della distribuzione normale invece è stato utilizzato l'intervallo notevole:

$$\left(\frac{(n-1) S_n^2}{\chi_{n-1, \alpha/2}^{2,+}}, \frac{(n-1) S_n^2}{\chi_{n-1, \alpha/2}^{2,-}} \right)$$

dove:

- S_n^2 è la varianza campionaria di una distribuzione normale da cui vengono fatti n campionamenti.
- $\chi_{n-1, \alpha/2}^{2,+}$: è il valore critico della coda superiore di livello $\alpha/2$ di una distribuzione χ^2 con $n-1$ gradi di libertà.
- $\chi_{n-1, \alpha/2}^{2,-}$: è il valore critico della coda inferiore di livello $\alpha/2$ di una distribuzione χ^2 con $n-1$ gradi di libertà.

Per definizione, data una variabile aleatoria X , il valore critico della coda inferiore di livello α di X è il minimo α -quantile. Rispettivamente, il valore critico della coda superiore di livello α di X è il massimo α -quantile.